

Forecasting the volatility of a stock market index; A hybrid approach integrating a GARCH model into various ANNs

Module Code: 24MAC302

Elizabeth White

Supervised by Axel Finke and Peng Liu

8th May 2025

Abstract

Volatility permeates a multitude of areas within finance, such as risk management and derivative pricing. While commonly used, traditional econometric models often fail to capture many of the stylised facts about volatility. As such, a large body of research has investigated the use of neural networks to improve forecasting accuracy. This thesis examines the effectiveness of the hybrid GARCH-DFN and GARCH-LSTM models in forecasting the volatility of the S&P 500 index, in comparison to the GARCH model and stand-alone feed-forward and LSTM networks. Additionally, this thesis investigates the impact of including log returns as an additional feature into the neural networks and assesses the performance of the GARCH model when using expanding compared to rolling windows. This thesis concludes that the hybrid GARCH-DFN and GARCH-LSTM offer significant improvements over both the GARCH model and the respective stand-alone ANNs. Specifically, the GARCH-LSTM offers a 33% and 17% reduction in HMSE compared to the GARCH(1,1) and LSTM models, respectively. Finally, the implications of the results are discussed within the context of the field of volatility forecasting and practical applications for the GARCH and neural networks are provided.

Acknowledgments

I would like to thank my supervisors, Dr Axel Finke, for his guidance during my initial research process, and Dr Peng Liu, for helping with the progression of my final work. Their feedback was pivotal in shaping my work and taught me a tremendous amount about academic research during the course of my thesis.

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Related Work and Literature	4
1.3	Problems and Research Question	6
1.4	Overview	6
2	Theoretical Background	7
2.1	Time Series	7
2.1.1	Statistical Properties	7
2.1.2	Statistical Tests	9
2.2	Financial Time Series	10
2.3	GARCH Models	10
2.3.1	ARCH Models	10
2.3.2	GARCH Models	11
2.3.3	Maximum Likelihood Estimation	11
2.4	Neural Network Models	12
2.4.1	Multilayer Perceptrons	12
2.4.2	Long Short Term Memory Models	16
3	Data	20
3.1	Data	20
3.1.1	Descriptive Statistics	20
4	Experiment	23
4.1	Train, validation and test splits	23
4.2	Volatility Measures	24
4.3	Evaluation of Models	25
4.4	Benchmark Model	25
4.5	Forecasting using GARCH models	26
4.5.1	Pre-Diagnostic Tests	26
4.5.2	Model Selection	27
4.5.3	Rolling and Expanding Window Scheme	28
4.6	Forecasting using Neural Networks	29
4.6.1	Proposed Hybrid Models	29
4.6.2	Feature Engineering and Data Preprocessing	29
4.6.3	Model Selection and Validation	30
4.6.4	Out-of-Sample Forecasting	32

5	Results and Discussion	34
5.1	Results	34
5.2	Discussion	38
6	Conclusion	40
6.1	Conclusions	40
6.2	Future Work	41
A	Appendix	43
A.1	Choice of Window Size for Volatility Proxy	43
A.2	Choice of Loss Function and Batch Size	44

Chapter 1

Introduction

1.1 Motivation

Volatility can be thought of as the magnitude of fluctuations over time. In financial markets, volatility is the measure of the dispersion of an asset’s returns over time. Typically, an asset’s volatility is quantified by its standard deviation over discrete time intervals, however it can also be defined over infinitesimal time intervals [1].

Within finance, forecasting the volatility of market variables is crucial across a multitude of areas. For investors, volatility is an important variable within models that price derivative securities, such as the Black-Scholes option pricing model [2], as well as in models that determine portfolio asset allocation.

Forecasting volatility is also central to risk management, in which financial institutions may use volatility to assess the downside risk of investments and manage their leverage, ensuring preparation for potential fluctuations within financial markets. Moreover, regulators use volatility to establish capital requirements which is important to ensure financial institutions maintain adequate liquidity to handle unexpected losses and continue operations during periods of market stress. Additionally, policy makers will use it to assess the strength of financial markets and the broader economy [1].

Due to its pertinence across finance, there has grown to be a large volume of research exploring the modelling and forecasting of the volatility of financial time series. With advancements being made in deep learning, particularly with the introduction of the Long Short-Term Memory model, there is a growing interest in investigating the potential benefits of integrating the forecasts of traditional econometric approaches within deep learning models.

1.2 Related Work and Literature

Volatility within financial markets has several well-documented stylised facts. One such fact is “volatility clustering” which is often present in financial time series. First documented by Mandelbrot and in various studies thereafter, volatility clustering is a phenomenon wherein periods of high volatility follow high volatility periods, and low volatility periods follow low volatility periods [3]. While frequently used ordinary least square models and traditional moving average and autoregressive models all assume *homoscedasticity*, meaning the variance of the error term is constant, the aforementioned models fail to capture this dependence structure within financial time series [4].

The Generalised Autoregressive Conditional Heteroscedasticity (GARCH) model, introduced by Bollerslev, is a commonly used econometric model which is *heteroscedastic* as it models the error term as having a time-dependent variance [5]. Many studies have shown that the GARCH model can be effective at modelling volatility. Akgiray found that the GARCH model outperformed benchmark forecasts and was superior to the exponentially weighted moving average model (EWMA) for modelling the monthly return variance of stock indices [6]. Additionally, when confined to the equity market, Figlewski found the GARCH model was superior to benchmark models when using daily data to forecast daily and monthly returns over short time horizons [7].

However, the GARCH model has several flaws [8]. The model has a strict functional form and imposes certain parameter constraints and distributional assumptions which have been demonstrated to not always be suitable [9] [10]. Additionally, the squared terms do not allow the model to account for the asymmetric impact of the returns, an additional stylised fact about volatility, wherein negative returns cause higher volatility than positive returns [11]. This rigidity of the GARCH model can hinder its performance when modelling complex, non-linear patterns that are typically present in financial time series.

As such, there is a large volume of research regarding the application of artificial neural networks (ANNs) within financial time-series forecasting. These models are capable of capturing complex, non-linear relationships without requiring strict distributional assumptions. Donaldson and Kamstra introduced a hybrid model which integrated a multilayer perceptron (MLP) into the GARCH framework, demonstrating its superior ability to capture asymmetric effects within financial time series compared to the GARCH model [12]. Subsequent studies have explored both hybrid and stand-alone feed-forward networks and have reported improved forecasting capabilities when forecasting the volatility of stock indices and commodity prices [13] [14] [15].

Variations of ANNs exist which have been able to reduce the forecasting error of financial time series. One such extension, the recurrent neural network (RNN), uses cyclical connections between neurons to allow the network to handle sequential data, such as time series. The Long Short Term Memory model, a class of recurrent neural networks (RNN), was designed to address the issue of diminishing or exploding gradients within standard RNN when learning long sequences of data [16]. Kim and Won utilised a three-layer LSTM on forecasting the volatility of the KOSPI index and demonstrated the network's ability to significantly improve forecast accuracy when compared to simple feed-forward neural networks and the GARCH model [17].

As mentioned, there is a substantial amount of research investigating the use of hybrid GARCH-type ANN models for various market variables, wherein the output from GARCH models is passed in as input to the ANN [14] [17] [18] [19]. Mademlis and Dritsakis demonstrated that a hybrid EGARCH-ANN model yielded a lower forecast error for the volatility of the FTSE MIB index compared to all GARCH-type models, in addition to the stand-alone ANN [20]. Kim and Won [17] and Kakade et al [21], both showed the superiority of a hybrid GARCH-LSTM for forecasting the volatility of assets within the equity and commodity markets.

However, not all research supports the use of ANNs and hybrid GARCH-ANN models, as several studies find that ANNs are prone to overfitting and struggle to generalise to new volatility regimes. For instance, Monfared and Enke found that a hybrid GJR-GARCH-ANN model performed better than the GJR-GARCH model during high volatility periods due to additional model complexity, but for low volatility periods, this additional model

complexity hindered the model’s ability to reduce the forecast error compared to the GJR-GARCH [22].

1.3 Problems and Research Question

The literature review highlights several problems in which this thesis aims to address. While there is a substantial amount of evidence that ANNs outperform the GARCH model, it is still uncertain if hybrid-ANNs can substantially improve the performance of stand-alone ANNs. Moreover, none of the aforementioned papers assess the performance of these models past March 2020. It remains unclear how a model fit on an additional major stock market crash and forecasted over the relatively low volatility period thereafter will perform. Therefore, this thesis seeks to explore *whether hybrid GARCH-ANNs can significantly improve the forecasting accuracy of ANNs and traditional GARCH models, particularly during low volatility periods*. A secondary focus of this thesis entails the investigation of whether the inclusion of the log returns as a feature in the neural networks can provide any additional improvement to the model.

Accordingly, this thesis will address such matters by fitting the GARCH, deep feed-forward (DFN) and LSTM networks along with the respective hybrid models to the last 25 years of S&P 500 returns data and compare the forecasting accuracy of all models over last four years.

1.4 Overview

The thesis is structured as follows. In Chapter 2 a brief overview of time series analysis, autoregressive conditional heteroskedastic models and feed-forward and recurrent neural networks used within this experiment is given. The data used to fit the models and evaluate their performance on is detailed in Chapter 3. The methodology¹ used to construct, select and ensure a robust evaluation of the forecasting accuracy of all the models is outlined in Chapter 4. Chapter 5 outlines and discusses the implications of the results found in the experiment with respect to the broader field. Chapter 6 summarises the results and suggests improvements for future work.

¹All code for this project can be found within the following Github repository <https://github.com/lizzywhite1/Hybrid-GARCH-LSTM-Forecasting-Thesis.git>

Chapter 2

Theoretical Background

The following section provides an overview of the theoretical material used throughout this thesis. Section 2.1 provides a brief overview of time series analysis and the statistical tests and properties used to analyse time series data. Section 2.2 and 2.3 provides a brief overview of the ARCH and GARCH models and explains how the returns process used in these models is found. Finally, Section 2.4 overviews the two deep learning models used within this thesis, the Multilayer Perceptron and the Long Short Term Memory model.

2.1 Time Series

A time series is a sequence of random variables, $x_1, x_2, \dots, x_T$, where x_1 represents the value of the series at the first time index, x_2 at the second, and so forth. A collection of random variables, $\{x_t\}_{t=1}^T$, indexed by t is known as a *stochastic process*. Hence, a time series can be regarded as one realisation of a stochastic process.

Definition 2.1.1. *A time series, $\{x_t\}_{t=1}^T$ is said to be weakly stationary (often referred to as stationary) if it satisfies the following properties:*

- *Mean stationary:* $\mathbb{E}[x_t] = \mu$ is constant for all t .
- *Covariance stationary:* For any $t, s \in \{1, \dots, T\}$, the covariance satisfies $\text{Cov}[x_t, x_s] = \gamma(|t - s|)^1$

2.1.1 Statistical Properties

Skewness is a measure of the asymmetry of the distribution of a random variable about its mean.

Definition 2.1.2. *The skewness of a random variable X is its third standardised moment, and is defined as*

$$\tilde{\mu}_3 = \mathbb{E} \left[\left(\frac{X - \mu}{\sigma} \right)^3 \right] = \frac{\mu_3}{\sigma^3}, \quad (2.1)$$

where μ is the mean, σ is the standard deviation, and μ_3 is the third central moment of X .

¹Covariance stationary also implies a weakly stationary time series is *variance stationary*, meaning $\text{Var}[x_t] = \sigma^2$ is constant for all t [23].

In this study, the Fisher–Pearson coefficient of skewness is used to calculate the sample skewness. It is defined as

$$g_1 = \frac{m_3}{m_2^{3/2}} = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^3}{\left(\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2\right)^{3/2}}, \quad (2.2)$$

where n is the number of samples and m_2 and m_3 are the second and third sample central moments, respectively.

The kurtosis of a distribution is a measure of the heaviness of the tails of a random variable, relative to its mean.

Definition 2.1.3. *The kurtosis of a random variable, X , is its fourth standardised moment, defined as*

$$Kurt[X] = \mathbb{E} \left[\left(\frac{X - \mu}{\sigma} \right)^4 \right] = \frac{\mu_4}{\sigma^4} \quad (2.3)$$

where μ_4 is the fourth-central moment of X .

The sample kurtosis is often calculated as

$$K = \frac{m_4}{m_2^2} = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^4}{\left(\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2\right)^2}.$$

However, in this study, the *excess kurtosis* is used. It is defined as

$$g_2 = \frac{m_4}{m_2^2} - 3 = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^4}{\left(\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2\right)^2} - 3 \quad (2.4)$$

where m_4 is the fourth sample central moment. As the kurtosis of the standard normal distribution is 3, this definition centres the measure such that a positive kurtosis ($g_2 > 0$) indicates a distribution with heavier tails and a negative kurtosis ($g_2 < 0$) indicates a distribution with lighter tails than a standard normal distribution [24].

The correlation is a measurement of the strength of the linear relationship between two variables. For a sample $\{(x_1, y_1), \dots, (x_n, y_n)\}$ of size n , the sample *Pearson's correlation coefficient* is computed as

$$r_{xy} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}, \quad (2.5)$$

where $\bar{x}$ and $\bar{y}$ are the sample means of x and y , respectively.

Similar to correlation, the *autocorrelation* (also known as the serial correlation) refers to the strength of the linear relationship between a variable and its lagged values. The lagged values of a time series at time t refer to the observations at previous time steps.

Definition 2.1.4. *The lag- k sample autocorrelation of a time series $\{x_t\}_{t=1}^T$ is defined as*

$$\hat{\rho}_k = \frac{\sum_{i=1}^{T-k} (x_i - \bar{x})(x_{i+k} - \bar{x})}{\sum_{i=1}^T (x_i - \bar{x})^2}. \quad (2.6)$$

An autocorrelation function (ACF) plot displays the sample autocorrelation at various lags. The following formula for the confidence interval is used

$$\hat{\rho}_k \pm \frac{z_{1-\alpha/2}}{\sqrt{T}}, \quad (2.7)$$

where $z_{1-\alpha/2}$ is the $(1 - \alpha/2)$ quantile of the standard normal distribution [24].

2.1.2 Statistical Tests

The *Jarque-Bera test* is a statistical test used to check whether a distribution has the skewness and kurtosis matching that of a normal distribution.

Definition 2.1.5. *The Jarque-Berra test statistic is defined as*

$$JB = \frac{n}{6}(g_1^2 + \frac{g_2^2}{4}). \quad (2.8)$$

If the data is normally distributed, the Jarque-Bera test statistic asymptotically has a chi-squared distribution with two degrees of freedom. The associated p-value is given by

$$p = 1 - F_{\chi_2^2}(JB), \quad (2.9)$$

where $F_{\chi_2^2}$ is the cumulative distribution function of the chi-squared distribution with two degrees of freedom. The null hypothesis is rejected with a significance level, α , if $p < \alpha$.

The *Ljung-Box test* is frequently used in econometrics to test for the absence of serial autocorrelation in the residuals of a time series model up to a specified lag. If there is no statistically significant autocorrelation, the model is said to not exhibit a significant lack of fit, and the null hypothesis is accepted, otherwise, the model exhibits lack of fit.

Definition 2.1.6. *For h number of lags, the Ljung-Box test statistic is given by*

$$Q = n(n+2) \sum_{k=1}^h \frac{\hat{\rho}_k^2}{(n-k)}, \quad (2.10)$$

where n is the number of samples and $\hat{\rho}_k$ is the sample autocorrelation at lag k .

If the null hypothesis holds, the test statistic Q asymptotically follows a chi-squared distribution with h degrees of freedom. Rejection of the null hypothesis with significance level, α , occurs if

$$Q > \chi_{1-\alpha, h}^2, \quad (2.11)$$

where $\chi_{(1-\alpha, h)}^2$ is the $(1 - \alpha)$ -quantile of the chi-squared distribution with h degrees of freedom [24].

The *Augmented Dickey-Fuller (ADF) test* is used to test if a time series has a unit root. The presence of a unit root in a time series indicates that it is non-stationary. If the null hypothesis of a unit root is rejected, this implies the time-series is stationary. This is equivalent to testing if $\gamma = 0$ in the following model equation

$$\Delta y_t = \alpha + \gamma y_{t-1} + \sum_{i=1}^{p-1} \delta_i \Delta y_{t-i} + \varepsilon_t, \quad (2.12)$$

where p is the lag of the autoregressive process, $\Delta y_t = y_t - y_{t-1}$ and $\varepsilon_t \sim \text{WN}(0, \sigma^2)$ is a white noise process. Equation (2.12) is a reparametrisation of an $AR(p)$ model in terms of first differences, where γ is the characteristic polynomial of the $AR(p)$ process. If $\gamma < 0$ this implies the time series is stationary.

The test statistic is given by

$$\hat{\tau}_{\gamma=0} = \frac{\hat{\gamma}}{SE(\hat{\gamma})} \quad (2.13)$$

where $\hat{\gamma}$ is the estimated coefficient on y_{t-1} and $SE(\hat{\gamma})$ is the standard error of the estimated coefficient, both obtained from the ordinary least squares regression of Equation 2.12. Under the null hypothesis, the distribution of the test statistic follows the Dickey-Fuller distribution. The null hypothesis is rejected if the test statistic is less than the critical value from the Dickey-Fuller distribution.

2.2 Financial Time Series

In financial markets, the price of an asset at time t , denoted p_t , is a time series $\{p_t\}_{t \in T}$ that records the price of an asset over a given period. As this experiment aims to model the daily fluctuations in closing prices, the returns of an asset are used and can be defined as

$$r_t = \frac{p_t - p_{t-1}}{p_{t-1}}, \quad (2.14)$$

As in financial markets, the returns usually represent a small percentage change,

$$\Delta \log(p_t) = \log(1 + r_t) \approx r_t. \quad (2.15)$$

When analysing financial time series, the logarithmic returns are therefore often defined as

$$r_t = \ln \left(\frac{p_t}{p_{t-1}} \right). \quad (2.16)$$

Throughout the rest of this paper, returns will refer to the logarithmic returns defined in 2.16.

2.3 GARCH Models

In traditional econometric models such as ARIMA, the optimal forecast, defined as the one which minimises the variance of forecast errors, is given as the conditional mean. These models are assumed to be homoscedastic, meaning the variance of the error term is constant over time [23]. However, volatility clustering within financial time series implies the conditional variance of returns is non-constant, as large moves in the returns of the previous day will affect the next.

2.3.1 ARCH Models

To model time series exhibiting volatility clustering, the Autoregressive Conditional Heteroscedasticity (ARCH) model was introduced by Engle [25]. This model explicitly incorporates the time-dependent variance, where the conditional variance now has both a stochastic and time-dependent component. The ARCH(q) model is specified by the equations²,

$$r_t = \mu_t + \sigma_t z_t, \quad z_t \sim N(0, 1) \quad (2.17)$$

$$\epsilon_t = \sigma_t z_t, \quad \epsilon_t \mid \mathcal{F}_{t-1} \sim N(0, \sigma_t^2) \quad (2.18)$$

$$\sigma_t^2 = \omega + \alpha_1 \epsilon_{t-1}^2 + \dots + \alpha_q \epsilon_{t-q}^2. \quad (2.19)$$

²It is common practise to assume the standardised residual, z_t , follows a normal distribution. However, z_t can be any i.i.d. distribution with zero mean and unit variance.

Equation (2.17) defines the returns process, containing the conditional mean equation μ_t , and the error term, composed of the normally distributed standardised residual, z_t , and the conditional standard deviation, σ_t . Equation (2.18) states that the error term, conditional on the set of information available at $t - 1$, denoted $\mathcal{F}_{t-1}$, follows a normal distribution with zero mean and conditional variance σ_t^2 . Equation (2.19) specifies that this conditional variance depends on the previous q squared residuals of the return process. As the conditional variance must be positive, the ARCH model imposes the constraints that all estimated coefficients must be non-negative.

2.3.2 GARCH Models

There is often a strong dependence structure within volatility, implying that a high-order ARCH model is often required to capture this dependence structure [7]. This makes the model computationally expensive and prone to over-fitting.

To address these limitations, the Generalised ARCH (GARCH) model was proposed by Bollerslev, extending the ARCH model by including the lagged conditional variance [5]. For the GARCH(p,q) model, Equation (2.17) and (2.18) remain unchanged, while Equation (2.19) is redefined as

$$\sigma_t^2 = \omega + \sum_{i=1}^q \alpha_i \epsilon_{t-i}^2 + \sum_{j=1}^p \beta_j \sigma_{t-j}^2 \quad (2.20)$$

where p is the number of lags used for the conditional variance.

As before, to ensure the conditional variance remains non-negative, the constraint that all parameters estimated must be non-negative is imposed. Additionally, to ensure the GARCH(p,q) process is weakly stationary, the following condition must hold

$$\sum_{i=1}^q \alpha_i + \sum_{j=1}^p \beta_j < 1. \quad (2.21)$$

2.3.3 Maximum Likelihood Estimation

The parameters of the GARCH model are estimated using maximum likelihood estimation. Let $\mathbf{r} = (r_1, \dots, r_T)^T$ be a weakly-stationary return series and $\mathcal{F}_{t-1}$ be the information set available at time $t - 1$. For a GARCH(1,1) model, the joint probability density function given the set of parameters, $\boldsymbol{\theta} = (\mu, \omega, \alpha_1, \beta_1)$, can be factorised as

$$f(\mathbf{r}; \boldsymbol{\theta}) = f(r_{k+1}, \dots, r_T \mid \mathcal{F}_k; \boldsymbol{\theta}) f(r_1, \dots, r_k; \boldsymbol{\theta}) \quad (2.22)$$

$$= \left(\prod_{t=k+1}^T f(r_t \mid \mathcal{F}_{t-1}; \boldsymbol{\theta}) \right) f(r_1, \dots, r_k; \boldsymbol{\theta}) \quad (2.23)$$

$$= \left(\prod_{t=k+1}^T \frac{1}{\sqrt{2\pi\sigma_t^2}} \exp \left(-\frac{(r_t - \mu)^2}{2\sigma_t^2} \right) \right) f(r_1, \dots, r_k; \boldsymbol{\theta}), \quad (2.24)$$

where $f(r_1, \dots, r_k; \boldsymbol{\theta})$ is the joint density for the initial k returns. For large sample sizes, the joint density for the initial k returns is usually ignored as it has little effect on the overall joint distribution. Equation (2.24) follows from Equation (2.17)-(2.18), where it

is assumed, that the standardised residuals are normally distributed. The conditional log-likelihood is given by

$$\ln L(\boldsymbol{\theta} \mid \mathbf{r}) = \sum_{t=1}^T \ln f(r_t \mid \mathcal{F}_{t-1}; \boldsymbol{\theta}) \quad (2.25)$$

$$= \sum_{t=1}^T -\frac{1}{2} \ln(2\pi) - \ln(\sigma_t^2) - \frac{(r_t - \mu_t)^2}{2\sigma_t^2}. \quad (2.26)$$

Using Equation (2.17)-(2.18) and Equation (2.20), the conditional variance of the GARCH(1,1) can be written as

$$\sigma_t^2 = \omega + \alpha_1(r_{t-1} - \mu_t)^2 + \beta_1\sigma_{t-1}^2. \quad (2.27)$$

The maximum likelihood estimate of the parameters is obtained by maximising the conditional log-likelihood in Equation (2.26), which involves finding the vector $\hat{\boldsymbol{\theta}}$ of parameters which satisfy

$$S(\hat{\boldsymbol{\theta}} \mid \mathbf{r}) = 0, \quad (2.28)$$

where $S(\boldsymbol{\theta} \mid \mathbf{r}) = 0$ is the *score vector*, defined as

$$S(\boldsymbol{\theta} \mid \mathbf{r}) = \begin{pmatrix} \frac{\partial \ln L(\boldsymbol{\theta} \mid \mathbf{r})}{\partial \mu_t} \\ \frac{\partial \ln L(\boldsymbol{\theta} \mid \mathbf{r})}{\partial \omega} \\ \frac{\partial \ln L(\boldsymbol{\theta} \mid \mathbf{r})}{\partial \alpha_1} \\ \frac{\partial \ln L(\boldsymbol{\theta} \mid \mathbf{r})}{\partial \beta_1} \end{pmatrix}. \quad (2.29)$$

As finding the analytical solution of the component of the score vector is typically infeasible, numerical optimisation algorithms must be used ³.

2.4 Neural Network Models

At its most fundamental level, an artificial neural network (ANN) is a network of nodes, or neurons, linked together by weighted connections. The aim of an ANN is to approximate a target function, $f : \mathbb{R}^p \rightarrow \mathbb{R}^m$ that maps an input vector, $\mathbf{x} \in \mathbb{R}^p$, to the corresponding output $\mathbf{y} \in \mathbb{R}^m$. The network defines a mapping $f(\mathbf{x}; \boldsymbol{\theta}) \in \mathbb{R}^m$, where $\boldsymbol{\theta}$ denotes the vector of parameters the network learns during training by minimising a chosen loss function, which quantifies the difference between $\mathbf{y}$ and $f(\mathbf{x})$ [27].

2.4.1 Multilayer Perceptrons

Multilayer perceptrons (MLPs), otherwise known as feed-forward networks, are a class of ANNs. The first layer of the MLP is known as the input layer, the last layer is known as the output layer. The layers in-between them are known as hidden layers. Each layer consists of a number of neurons, where the information flows in one direction, from input to output.

³This paper uses the Python “ARCH” package to do so [26].

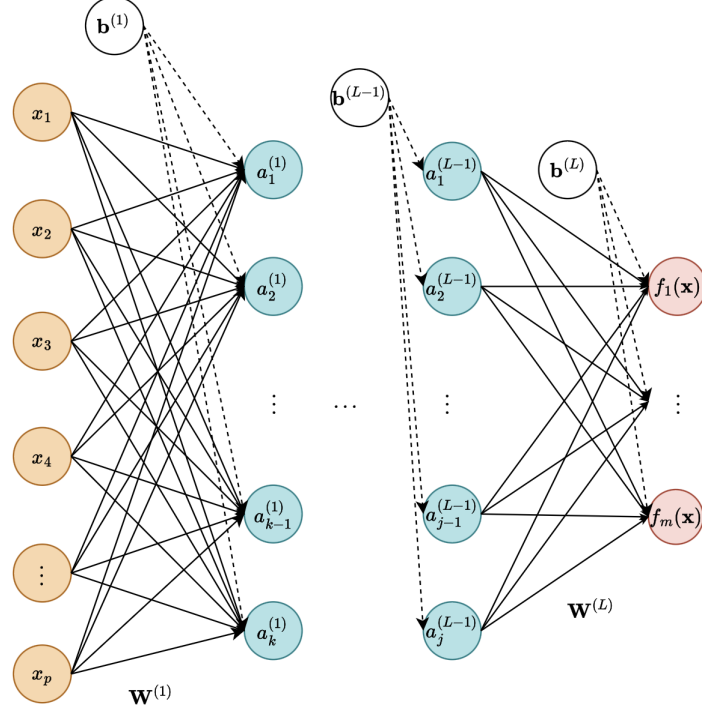


Figure 2.1: A general multi-layer perceptron with p input units and m output units. The network has multiple hidden units, with the first having k units and the last having j units. The matrices $\mathbf{W}^{(1)}$ and $\mathbf{W}^{(L)}$ represent the weighted connections for the first and last layers, and $\mathbf{b}^{(1)}$, $\mathbf{b}^{(L-1)}$ and $\mathbf{b}^{(L)}$ are the corresponding bias layers.

Forward Passes

Each neuron in the network stores a real number called an *activation*, denoted $a_j^{(l)}$, for neuron j in layer l . The activations are computed sequentially wherein the output of the $(l-1)^{th}$ layer is passed into the l^{th} layer using a weighted sum, followed by a non-linear activation function. This process of sequentially computing all activations is known as a *forward pass* and is illustrated in Figure 2.1.

Let $\mathbf{x} \in \mathbb{R}^n$, denote the input vector which initialises the input layer of the network. The matrix of weights connecting the $(l-1)^{th}$ layer with k neurons to the l^{th} layer with j neurons is denoted as $\mathbf{W}^{(l)} \in \mathbb{R}^{j \times k}$, with entries $w_{jk}^{(l)}$. The bias vector for the l^{th} layer is denoted $\mathbf{b}^{(l)} \in \mathbb{R}^j$ and $g^{(l)}(\cdot)$ is the activation function for that layer. A single activation for neuron j within layer l can be expressed as

$$a_j^{(l)} = g^{(l)}(z_j^{(l)}) \quad (2.30)$$

$$z_j^{(l)} = \left(\sum_k w_{jk}^{(l)} a_k^{(l-1)} \right) + b_j^{(l)} \quad (2.31)$$

where $z_j^{(l)}$ is called the network input for neuron j in layer l . The activations for all neurons within layer l is given by

$$\mathbf{A}^{(l)} = g^{(l)}(\mathbf{Z}^{(l)}) \quad (2.32)$$

$$\mathbf{Z}^{(l)} = \mathbf{W}^{(l)} \mathbf{A}^{(l-1)} + \mathbf{b}^{(l)} \quad (2.33)$$

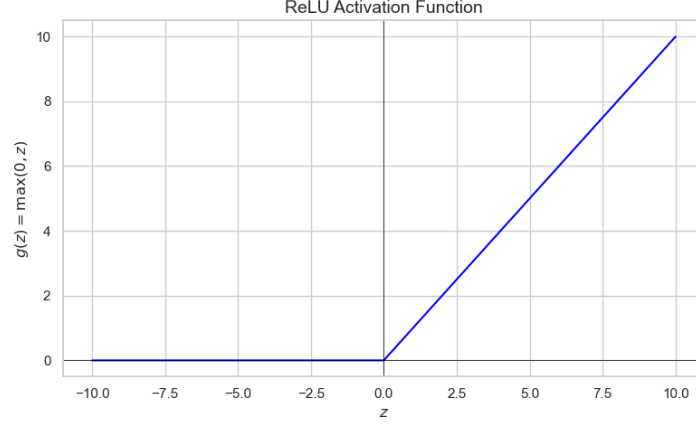


Figure 2.2: The ReLU function over the interval $[-10, 10]$.

where $\mathbf{Z}^{(l)}$ will be called the network input for layer l . For clarity, $\mathbf{Z}^{(l)}$ can be written component wise

$$\mathbf{Z}^{(l)} = \begin{bmatrix} w_{11}^{(l)} & \dots & w_{1k}^{(l)} \\ \vdots & \ddots & \vdots \\ w_{j1}^{(l)} & \dots & w_{jk}^{(l)} \end{bmatrix} \begin{bmatrix} a_1^{(l-1)} \\ \vdots \\ a_k^{(l-1)} \end{bmatrix} + \begin{bmatrix} b_1^{(l)} \\ \vdots \\ b_j^{(l)} \end{bmatrix}. \quad (2.34)$$

The activation function, $g : \mathbb{R} \rightarrow \mathbb{R}$, is a non-linear function that transforms the input from the previous layer. Non-linear activations are essential within MLPs as they allow the network to capture complex, non-linear mappings. In modern neural networks, it is usually recommended to use the rectified linear unit, or ReLU, function, defined as

$$g(z) = \max\{z, 0\}. \quad (2.35)$$

The ReLU function can be seen in Figure 2.2.

Using the input layer as the initial activation, Equation (2.33) is applied recursively through all the MLP's layers. For a network with L layers, the final mapping $f(\mathbf{x})$ is computed as

$$f(\mathbf{x}) = g^{(L)}(\mathbf{W}^{(L)} g^{(L-1)}(\dots g^{(2)}(\mathbf{W}^{(2)} g^{(1)}(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)}) \dots) + \mathbf{b}^{(L)}). \quad (2.36)$$

Loss Functions

To quantify how well the model has approximated the function f , neural networks use loss functions, denoted as $\mathcal{L} : \mathbb{R}^m \rightarrow \mathbb{R}$, to measure the distance between the two values. There is a plethora of loss functions which could be used to train a neural network depending on the task and data at hand [28] [29].

One such loss function is the mean absolute error (MAE), is defined as

$$\mathcal{L}(\boldsymbol{\theta}) := \frac{1}{N} \sum_{i=1}^N \|f(\mathbf{x}^{(i)}; \boldsymbol{\theta}) - \mathbf{y}^{(i)}\|_1 = \frac{1}{N} \sum_{i=1}^N \ell_1(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), \mathbf{y}^{(i)}) \quad (2.37)$$

where $\|\cdot\|_1$ is the usual $L1$ norm and

$$\ell_1(f(\mathbf{x}^{(i)}), \mathbf{y}^{(i)}) = \|f(\mathbf{x}^{(i)}; \boldsymbol{\theta}) - \mathbf{y}^{(i)}\|_1 \quad (2.38)$$

is the absolute error for one training sample.

Gradient Descent with Backpropagation

To find the parameters which minimise the loss function for a given training input, an optimisation algorithm, known as *gradient descent*, is used. The goal of gradient descent is to find a local or global minima of the loss function, expressed as

$$\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}). \quad (2.39)$$

Assuming an initial parameter guess, $\boldsymbol{\theta}^{(0)}$, has been chosen, gradient descent can be formalised as

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}^{(t)}) \quad (2.40)$$

$$= \boldsymbol{\theta}^{(t)} - \eta \frac{1}{N} \sum_{i=1}^N \nabla_{\boldsymbol{\theta}} \ell_1(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}^{(t)}), \mathbf{y}^{(i)}), \quad (2.41)$$

where t is the number of iterations, ℓ is the chosen point-wise loss, and $\eta > 0$ is the *learning rate*.

The learning rate is a hyperparameter which determines the step size taken in the direction of the steepest descent. While larger learning rates can accelerate convergence by allowing for more substantial steps to be taken in the direction of the local minima, they may cause the optimisation to diverge or oscillate around the local minima due to overshooting. Conversely, smaller learning rates tend to produce more stable convergences but often require more iterations (known as *epochs*) to reach an optimal solution.

Once the forward pass, in Equation (2.33), is completed, to find $\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$, an expression for the partial derivatives, $\frac{\partial \mathcal{L}}{\partial w_{jk}^{(l)}}$ and $\frac{\partial \mathcal{L}}{\partial b_j^{(l)}}$, must be found. The method of finding these partial derivatives is known as a *backward pass*. An adapted method by Nielsen is used to present an efficient way of computing the partial derivatives [30].

First, an intermediate term, known as error at the j^{th} neuron in layer l , is defined as

$$\delta_j^{(l)} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}}. \quad (2.42)$$

This term is the magnitude of the impact that this neuron has on the overall loss function. The vector of all the errors within a layer will be denoted as $\boldsymbol{\delta}^{(l)}$.

It follows that the error at the output layer, L , for the j^{th} neuron can be computed as

$$\delta_j^{(L)} = \frac{\partial \mathcal{L}}{\partial z_j^{(L)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(L)}} \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} = \frac{\partial \mathcal{L}}{\partial a_j^{(L)}} (g^{(L)})'(z_j^{(L)}), \quad (2.43)$$

which follows directly from Equation (2.30).

Equation (2.43) can be used to initialise a recursive relationship for the error, $\delta_j^{(l)}$, of any neuron in an earlier layer, given by

$$\delta_j^{(l)} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \quad (2.44)$$

$$= \sum_k \frac{\partial \mathcal{L}}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}} \quad (2.45)$$

$$= \sum_k \delta_k^{(l+1)} \frac{\partial z_k^{(l+1)}}{\partial z_j^{(l)}}, \quad (2.46)$$

Using Equation (2.31) and (2.30) respectively, $z_j^{(l+1)}$ can be written as

$$z_k^{(l+1)} = \sum_j w_{kj}^{(l+1)} a_j^{(l)} + b_k^{(l+1)} = \sum_j w_{kj}^{(l+1)} g^{(l)}(z_j^{(l)}) + b_k^{(l+1)}. \quad (2.47)$$

Hence, Equation (2.46) becomes

$$\delta_j^{(l)} = \sum_k \delta_k^{(l+1)} w_{kj}^{(l+1)} (g^{(l)})'(z_j^{(l)}), \quad (2.48)$$

which can also be written using matrix notation

$$\boldsymbol{\delta}^{(l)} = \left((\mathbf{W}^{(l+1)})^T \boldsymbol{\delta}^{(l+1)} \right) \odot (g^{(l)})'(\mathbf{Z}^{(l)}) \quad (2.49)$$

where $\odot$ is the Hadamard product.

Using Equation (2.31) one can obtain the desired partial derivatives,

$$\frac{\partial \mathcal{L}}{\partial b_j^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial b_j^{(l)}} = \delta_j^{(l)} \quad (2.50)$$

and,

$$\frac{\partial \mathcal{L}}{\partial w_{jk}^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial w_{jk}^{(l)}} = \delta_j^{(l)} a_k^{(l-1)}. \quad (2.51)$$

The process of computing the partial derivatives of the loss function with respect to the model's parameters using a forward and backward pass is known as *backpropagation*. It is a fundamental procedure in the training of any neural network. Using the forward and backward passes, the gradient descent algorithm⁴ with backpropagation is summarised in Algorithm (1).

2.4.2 Long Short Term Memory Models

A recurrent neural network (RNN) is a class of ANN, which allows for cyclical connections, making it capable of handling sequential data by using both the input at the current time step in addition to the activations from previous time steps.

The Long Short Term Memory model (LSTM), is a type of RNN, designed to address the *vanishing gradient problem* [16]. As the number of time-steps increase within the network, the recursive chain of partial derivatives which are computed during backpropagation increases. A gradient less than one ultimately causes the gradient to vanish, hindering its ability to learn long term dependencies in data.

The LSTM fixes this problem by introducing a *cell state* which holds information across time steps, along with *gated units* which determine the percentage of information to flow in and out of the cell. By selectively maintaining and updating its memory, the LSTM is better suited at handling long-term dependencies within data.

⁴In practise, popular optimisers such as Adam, use Stochastic Gradient Descent to optimise parameters [31]. This involves updating the parameters using a small batch of samples rather than the whole training set.

Algorithm 1 Gradient Descent with Backpropagation

Require: Training data (X, Y) with N samples, learning rate η , number of epochs E .

Randomly initialise weights $\mathbf{W}^{(l)}$ and biases $\mathbf{b}^{(l)}$ for all layers $l = 1, \dots, L$.

for epoch = 1 to E **do**

for each training example $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$ in (X, Y) **do**

Forward Pass:

 Set input activation $\mathbf{A}^{(0)} \leftarrow \mathbf{x}^{(i)}$

for each layer $l = 1$ to L **do**

 Compute network input: $\mathbf{Z}^{(l)} \leftarrow \mathbf{W}^{(l)} \mathbf{A}^{(l-1)} + \mathbf{b}^{(l)}$

 Compute activation: $\mathbf{A}^{(l)} \leftarrow g^{(l)}(\mathbf{Z}^{(l)})$

end for

Backward Pass:

 Compute layer error: $\delta^{(L)} \leftarrow \nabla_{\mathbf{A}^{(L)}} \mathcal{L} \odot g'(\mathbf{Z}^{(L)})$

for each layer $l = L - 1$ down to 1 **do**

$\delta^{(l)} \leftarrow (\mathbf{W}^{(l+1)})^T \delta^{(l+1)} \odot g'(\mathbf{Z}^{(l)})$

end for

Gradient Descent:

for each layer $l = 1$ to L **do**

$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \frac{\eta}{N} \sum_x \delta^{(l,x)} (\mathbf{A}^{(l-1,x)})^T$.

$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \frac{\eta}{N} \sum_x \delta^{(l,x)}$.

end for

end for

end for

return Final parameter set $\theta := \{\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \dots, \mathbf{W}^{(L)}, \mathbf{b}^{(L)}\}$

Forward Pass

Let h be the dimension of the hidden and cell states, and d be the number of input features. The following variables are defined:

- $\mathbf{x}_t \in \mathbb{R}^d$ denotes the input at time t . Noting that the subscript notation is now used to denote time steps rather than individual neurons.
- $\mathbf{h}_t \in \mathbb{R}^h$ is the output of the LSTM cell at time t , known as the *hidden state*. It encodes the network's "short-term" memory.
- $\mathbf{c}_t \in \mathbb{R}^h$ is the cell state at time t . It holds the "long-term" memory of the network.
- $\bar{\mathbf{c}}_t \in \mathbb{R}^h$ is the cell input activation vector, also known as *cell candidate*, at time t .
- $\mathbf{W}_c, \mathbf{W}_i, \mathbf{W}_f, \mathbf{W}_o \in \mathbb{R}^{h \times d}$ are the input weight matrices for the cell candidate, input gate, forget gate and output gate, respectively.
- $\mathbf{R}_c, \mathbf{R}_i, \mathbf{R}_f, \mathbf{R}_o \in \mathbb{R}^{h \times h}$ are known as the *recurrent weight* matrices applied to the hidden state for the cell candidate and the corresponding gates.
- $\mathbf{b}_c, \mathbf{b}_i, \mathbf{b}_f, \mathbf{b}_o \in \mathbb{R}^h$ denote the bias vectors applied to the weighted sum for the cell candidate and the corresponding gates.

Once the hidden and cell state, h_0 and c_0 , have been initialised⁵, the first step in the

⁵In practise, it is common to initialise both states to zero at the start of each forward pass, however, they can also become learnable parameters to be optimised.

forward pass is to compute the *input gate*, computed as

$$\mathbf{i}_t = \sigma(\bar{\mathbf{i}}_t) \quad (2.52)$$

$$= \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{R}_i \mathbf{h}_{t-1} + \mathbf{b}_i), \quad (2.53)$$

where $\bar{\mathbf{i}}_t \in \mathbb{R}^h$ is the input gate's activation vector at time t and σ is the logistic sigmoid function. The logistic sigmoid function is defined as

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.54)$$

The *forget gate* is defined as

$$\mathbf{f}_t = \sigma(\bar{\mathbf{f}}_t) \quad (2.55)$$

$$= \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{R}_f \mathbf{h}_{t-1} + \mathbf{b}_f), \quad (2.56)$$

where $\bar{\mathbf{f}}_t$ is the activation vector for the forget gate at time t .

Using the Equation (2.53) and (2.56) the cell state is updated and becomes

$$\mathbf{c}_t = \mathbf{i}_t \odot \bar{\mathbf{c}}_t + \mathbf{f}_t \odot \mathbf{c}_{t-1}, \quad (2.57)$$

where $\bar{\mathbf{c}}_t$ is defined as

$$\bar{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{R}_c \mathbf{h}_{t-1} + \mathbf{b}_c). \quad (2.58)$$

Equation (2.57) gives an insight into the role of the input and forget gate. Noting the fact that the logistic sigmoid function is bounded by $(0, 1)$, the forget gate can be interpreted as the percentage of long-term memory, $\mathbf{c}_{t-1}$ to be remembered. Additionally, the input gate can be thought of as the percentage of potential memory, $\bar{\mathbf{c}}_t$, to be remembered.

The output gate is defined as

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{R}_o \mathbf{h}_{t-1} + \mathbf{b}_o). \quad (2.59)$$

Using Equation (2.59), the hidden state is updated

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh \mathbf{c}_t. \quad (2.60)$$

Similarly, Equation (2.60) implies that the output gate is used to determine the percentage of short-term memory the network should remember.

The operations in the LSTM cell can be seen in Figure 2.3, illustrating how the input vector and previous hidden state all contribute to the three gates and the cell candidate. Furthermore, one can see how the previous hidden state and the updated cell candidate then contribute to form the updated hidden state. The LSTM cell is a part of a broader layer of cells which compute the cell state and hidden state for each time-step. This chain structure is illustrated in Figure 2.4 and demonstrates how the hidden and cell states pass through the network, in addition to how the loss is calculated at every time step.

Backpropagation Through Time

The derivation of the backward pass for the LSTM follows a similar method as seen in Section 2.4.1. The process of finding these partial derivatives is known as *Backpropagation Through Time* (BPTT). The derivation of the BPTT algorithm is omitted as it is outside the scope and aim of this thesis.

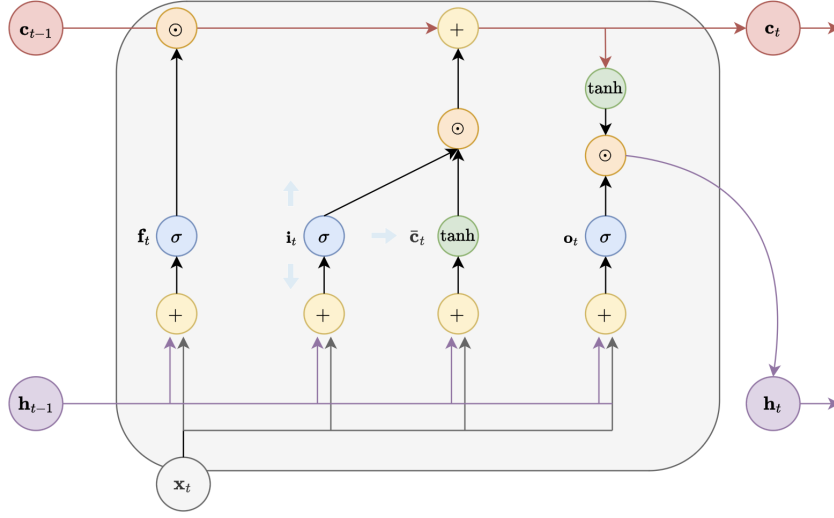


Figure 2.3: An illustration of one forward pass inside an LSTM cell. The operations $+$ and $\odot$ represent the weighted sums and Hadamard product of the inputs respectively.

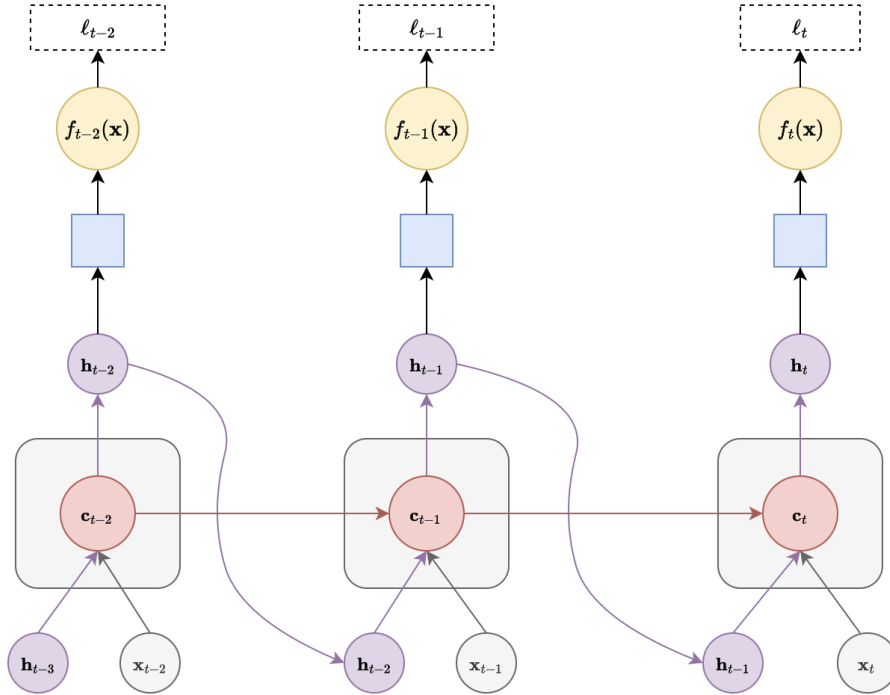


Figure 2.4: An “unrolled” LSTM network. This network has one layer and goes through three time steps. The empty blue square boxes represent possible additional layers, between the first LSTM layer and the final output such as additional LSTM layers or a layer of neurons. Inspired by [32].

Chapter 3

Data

This chapter describes the dataset used in this experiment and outlines its key statistical properties.

3.1 Data

The stock index selected is the S&P 500. The S&P 500 is a stock index that tracks the performance of the 500 largest companies traded on U.S. stock markets, covering approximately 80% of available market capitalisation. In addition to risk management and derivative pricing, forecasting the volatility of the S&P 500 is crucial for assessing US market sentiment as it is a key indicator of the strength of and sentiment regarding the US economy. For the S&P 500, 6,319 daily observations were obtained from Yahoo Finance covering the period of 1999-12-31 to 2025-02-13.

3.1.1 Descriptive Statistics

To forecast the daily realised volatility of the S&P 500, the two key variables used are the log returns process, defined in Equation (2.16), and the squared log returns process. Figure 3.1 displays the log returns over the whole sample period. Figure 3.1 illustrates the volatility clustering in the returns process, showing periods of significant change in daily closing prices tend to be followed by further big moves, and small moves tend to be followed by small ones.

Table 3.1 shows the descriptive statistics, specifically the mean, standard deviation, skewness and kurtosis, for the data used throughout this study. Additionally, the table also includes the Jarque-Bera test, a test for normality, and the Augmented Dickey-Fuller test, a stationarity test.

Name	Mean	Standard Deviation	Skewness	Kurtosis	Jarque-Bera	ADF
SPX Close	2129.77	1267.98	1.241	3.567	1705 (***)	2.98
SPX Returns	0.7353	26.38	-0.6899	17.16	53254 (***)	-15.78 (***)
SPX Log Return	0.0002257	0.01222	-0.3865	13.41	28664 (***)	-19.11 (***)
SPX Log Return Squared	0.0001493	0.0005252	14.01	291.9	22176906 (***)	-8.56 (***)

Note: Asterisks (***) indicate significance at the 1% level; (**) at 5%, and (*) at 10%.

Table 3.1: Summary statistics and normality and stationarity tests for the S&P 500 from 2000 to 2025.

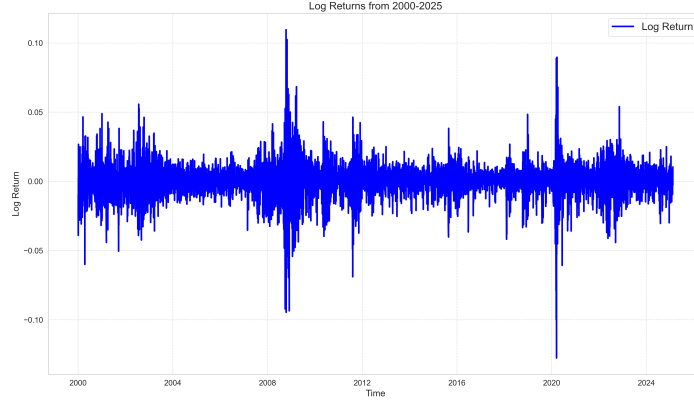


Figure 3.1: Log returns over 2000 to 2025.

Using the Augmented Dickey Fuller (ADF) test on the closing prices, the ADF test statistic is 2.98, which fails to reject the null hypothesis of the presence of a unit root at a 1% significance level, indicating non-stationarity. Therefore, the closing prices were converted to a stationary time series by taking the logarithmic difference in Equation (2.16). From the inspection of the log returns process in Figure 3.1 over the 2000 to 2025 period, the log return process appears to be stationary with zero mean. Table 3.1 confirms this, as the log returns have a mean of 2.257×10^{-4} and an ADF test which rejects the null hypothesis of a unit root at at 1% significance level. Figure 3.2 and Table 3.1, reveals the log returns are negatively skewed, indicating a more pronounced tail on the left of the distribution, likely due to the asymmetric impact of returns.

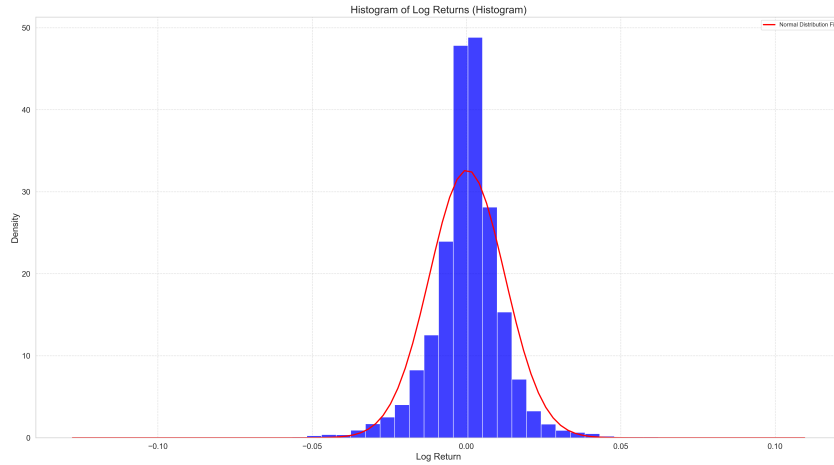


Figure 3.2: Density histogram of log returns of the S&P 500 over the period 2000 to 2025.

Note. The red line represents a normal distribution fitted to the data using the same mean and variance.

Additionally, the distribution has a kurtosis of 13.41, implying the distribution has a more pronounced peak and thicker tails relative to a normal distribution. Distributions with such a shape are known as *leptokurtic*, implying that compared to a normal distribution, there will be more extreme outliers from the mean. The assumption of non-normality of the log returns is confirmed with the Jarque-Bera test, as seen in Equation

(2.8), which a test statistic of 28,664, rejecting the null hypothesis of normality at a 1% significance level. The leptokurtic distribution of the log returns process is a key feature of volatility clustering.

Chapter 4

Experiment

This chapter outlines the experiment conducted in this thesis. Section 4.1 outlines the train, validation and test splits used in this paper to reduce overfitting. Section 4.3 outlines the loss functions used to assess the accuracy of the forecasts against the volatility measure detailed in Section 4.2. Additionally, Section 4.4 details the baseline model, used as a benchmark to evaluate all the models against. Lastly, Section 4.5 and Section 4.6 detail the model selection and forecasting process of all the models used in this paper.

4.1 Train, validation and test splits

The data was split into three sets as seen in Figure 4.1: the training, validation and test sets. The training set is used to fit all the models, while the validation set is used for out-of-sample forecasting to select the best hyperparameters and model architecture for the neural network. The test set is strictly reserved for the final out-of-sample forecasting. The distinction between the validation and test set is crucial for the unbiased assessment of the out-of-sample forecasting performance of all models. Since model selection is primarily determined by its performance on the validation set, it is important to confirm that the chosen model also generalises well to unseen data, namely the test set.

The training data encompasses the volatile periods of the “*dot-com bubble*” of 2000 to 2001, and the even more volatile 2008 financial crisis. Additionally, the validation set contains the volatile period of early 2020 during the start of COVID-19. However, the testing set from mid 2021 to early 2025 is characterised by fairly low volatility, aside from a moderately volatile period around the start of the Russo-Ukraine war. This is reflected in the standard deviation of the log returns. The combined train and validation set, in which all final models are fit on, has a standard deviation of 0.01247, while the standard deviation of the test set is 0.01066. The split is summarised in Table 4.1.

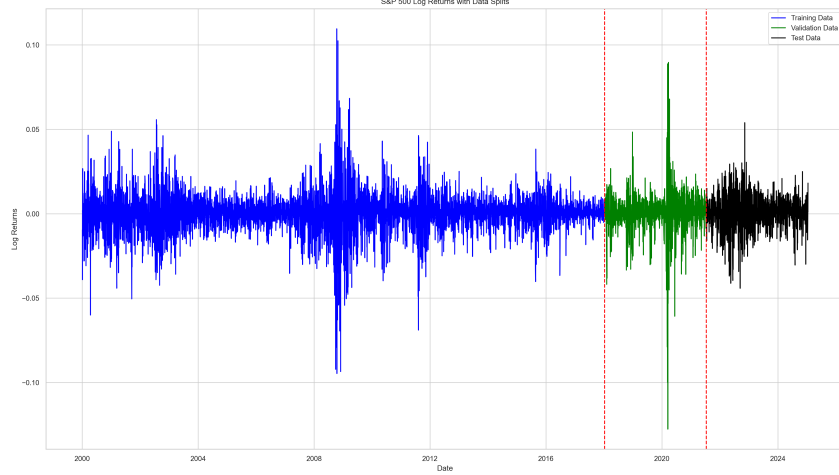


Figure 4.1: S&P 500 log returns with training, validation, and test splits in blue, green and black, respectively. The first red line indicates where the training set ends and the validation set begins. The second red line indicates where the validation set ends and the test set begins.

Data Split	Start Date	End Date	Observations	% of Total
Train	03/01/2000	09/01/2018	4534	72.00%
Validation	10/01/2018	13/07/2021	882	14.00%
Test	14/07/2021	15/01/2025	882	14.00%
Total	31/12/1999	13/02/2025	6298 ¹	100.00%

¹ 21 observations were lost due to the calculation of log returns and the target variable.

Table 4.1: The training, validation, and test set splits with the number of observations and their proportion of the total dataset.

4.2 Volatility Measures

As financial data is only available in discrete time intervals, but price fluctuations can occur continuously, true volatility is considered a *latent* variable, meaning it is unobservable. Consequently, there is a large body of research focused on finding a suitable *ex-post* volatility measure to estimate the true volatility [33] [34]. The choice of volatility proxy can have a significant impact on the model’s performance [35]. Some models may perform well when compared against the respective proxy but fail to capture the day-to-day movements of the asset.

Realised volatility is an *ex-post* measure of volatility, which uses previous price movements over a specific period to quantify how much the asset has fluctuated. The realised volatility is used as measure to evaluate the forecasts of the models against. In the absence of intra-day data, this thesis follows common practise and defines the realised volatility at time t to be

$$RV_t = \sqrt{\frac{1}{T} \sum_{i=t}^{t+T-1} (r_i - \bar{r})^2}, \quad (4.1)$$

where $\bar{r} = \frac{1}{T} \sum_{i=t}^{t+T-1} r_t$ is the average log return over T days [17][36]. This experiment uses a 21-day window, representing the approximate number of trading days in a month. Appendix A.1 further justifies this choice.

4.3 Evaluation of Models

In order to evaluate the forecast accuracy of all models, four loss functions are employed. The first is the mean absolute error (MAE) due to its interpretability. The second is the mean squared error (MSE), which penalises large deviations more heavily, making it useful for assessing model performance on outliers. The two metrics are defined as,

$$\text{MAE} = \frac{1}{T} \sum_{i=1}^T |\hat{\sigma}_t - \text{RV}_t| \quad (4.2)$$

$$\text{MSE} = \frac{1}{T} \sum_{i=1}^T (\hat{\sigma}_t - \text{RV}_t)^2, \quad (4.3)$$

where $\hat{\sigma}_t$ denotes the predicted volatility at time t and RV_t denotes the realised volatility.

Traditional loss functions, such as the MAE and MSE, are usually suited to linear models. Since both the GARCH and neural network are non-linear and the returns process is heteroskedastic, scale-dependent loss functions may misrepresent model performance, as they assign equal weight to errors regardless of current volatility regime [14]. To account for this, the heteroscedasticity-adjusted MAE and MSE are used and defined as

$$\text{HMAE} = \frac{1}{T} \sum_{i=1}^T \left| 1 - \frac{\hat{\sigma}_t}{\text{RV}_t} \right| \quad (4.4)$$

$$\text{HMSE} = \frac{1}{T} \sum_{i=1}^T \left(1 - \frac{\hat{\sigma}_t}{\text{RV}_t} \right)^2. \quad (4.5)$$

These metrics measure the absolute and squared percentage errors, respectively [37]. Since the HMAE and HMSE normalise the forecast error, they are scale-invariant and better reflect the performance of the model across varying volatility regimes.

4.4 Benchmark Model

To evaluate the performance of all models, a benchmark model is needed. Due to the serial correlation seen in volatility, a suitable method to generate baseline forecasts would be a historical moving average method [6]. The benchmark model takes the standard deviation over the previous T days to make a forecast for the next day, it is given as

$$\sigma_t = \sqrt{\frac{1}{T} \sum_{i=1}^T (r_{t-i} - \bar{r})^2}, \quad (4.6)$$

where T is the rolling window size and $\bar{r} = \frac{1}{T} \sum_{i=1}^T r_{t-i}$. For this experiment a 21-day window will be used, matching the window size of our target variable in Equation 4.1.

4.5 Forecasting using GARCH models

4.5.1 Pre-Diagnostic Tests

To motivate the use of a GARCH model, a first-order autoregressive model, AR(1), is fit to the data. The AR(1) model is defined as

$$r_t = \phi r_{t-1} + \epsilon_t, \quad (4.7)$$

where ϕ is the autoregressive coefficient and $\epsilon_t \sim WN(0, \sigma^2)$ is a white noise process. The conditional mean of the model depends on the previous observation and is given by $\mathbb{E}[r_t | \mathcal{F}_{t-1}] = \phi r_{t-1}$. Unlike the conditional mean, the conditional variance remains constant as $\text{Var}(r_t | \mathcal{F}_{t-1}) = \sigma^2$.

Applying the Ljung-Box test to the residuals gives a p-value of 0.88 at lag 1. At the 1% significance level, the null hypothesis of no autocorrelation is not rejected, implying that the AR(1) model adequately captures the conditional mean of the returns process.

However, when applying the Ljung-Box test to the squared residuals, significant autocorrelation is detected at lag 1, and subsequent lags, at the 1% significance level. Figure 4.2 shows the ACF plot of the squared residuals of the model, highlighting the strong dependence across lags. This suggests the presence of conditional heteroscedasticity in the return series and hence violates the assumption of constant variance, indicating the AR(1) model fails to capture this time-varying volatility. An improved approach would be to use a GARCH model, which explicitly models this conditional variance by incorporating past information into its forecast variance [25].

Additional analysis of the ACF plot for the squared returns, as seen in Figure 4.3, shows a significant dependence structure, indicating the suitability of a GARCH model over the ARCH model to minimise the number of parameters needed.

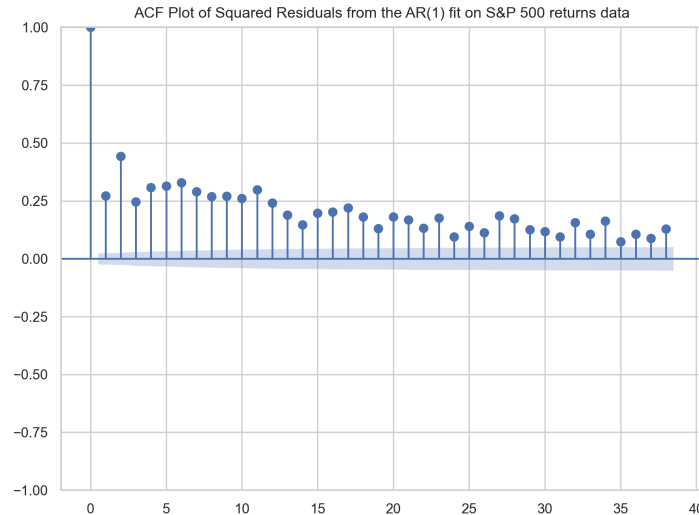


Figure 4.2: ACF plot of the squared residuals of the AR(1) model fit over the S&P 500 returns series from 2000 to 2025.

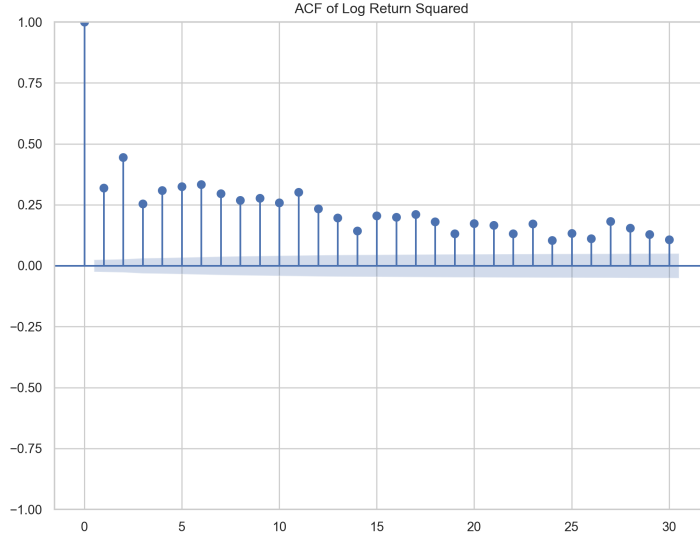


Figure 4.3: ACF plot of the squared log return of the S&P 500 from 2000 to 2025.

4.5.2 Model Selection

The GARCH(1,1) is selected as additional parameters are rarely required to capture the volatility clustering in the data [38]. Two different types of models are considered, the GARCH(1,1) with zero mean and the GARCH(1,1) with constant mean.

The GARCH(1,1) models are fit on the combined train and validation set, outlined in Table 4.1. The AIC and BIC of the GARCH(1,1) with a zero and constant mean are shown in Table 4.2. The GARCH(1,1) with constant mean has the lower AIC and BIC of 14896.2 and 14922.6. As the GARCH(1,1) with constant mean has the lower information criterion it is selected as the final model to be evaluated over the test set. The parameters estimated for the GARCH(1,1) with constant mean model are shown in Table 4.3.

GARCH Model	AIC	BIC
GARCH(1,1) with Zero Mean	14926.4	14946.2
GARCH(1,1) with Constant Mean	14896.2	14922.6

Table 4.2: AIC and BIC of the two considered GARCH models on the combined train and validation set.

GARCH Model	μ	ω	α_1	β_1
GARCH(1,1) with Constant Mean	0.0612	0.0235	0.1264	0.8571

Table 4.3: Estimated parameters on the combined train and validation set of the GARCH(1,1) with constant mean model.

4.5.3 Rolling and Expanding Window Scheme

The GARCH models were evaluated using a rolling and expanding window over the test set. The rolling window scheme, illustrated in Figure 4.4, is evaluated using a 252 day window, which is common practise as it is approximately one year of trading days. The rolling window fits the GARCH model from $t - 252$ to $t - 1$ days and makes a forecast for the next day on the test set. The first forecast corresponds to the first observation in the test set. The window then shifts forward by one day, and the process repeats until there is a corresponding forecast for every day in the test set.

The expanding window approach, as shown in Figure 4.5, begins by making a forecast for the first day in the test set. For each subsequent iteration, the expanding window extends by one observation, using all the data available prior to the forecasted day to make a forecast for the next day. The first window in the expanding window scheme represents all the information available prior to the first observation in the test set.

Using a rolling or expanding window prevents data leakage. If the model's coefficients were estimated on the entire dataset and then forecasts were retrospectively computed, this would allow the neural network to effectively “see into the future”. By using the rolling or expanding window approach, the neural network is never trained on data that would not be available at the current time step.

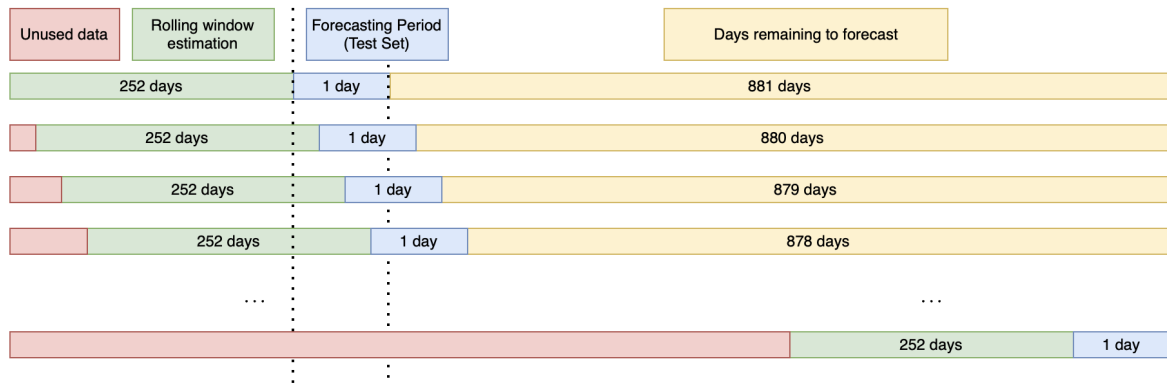


Figure 4.4: Rolling window scheme used for the GARCH model forecasts.

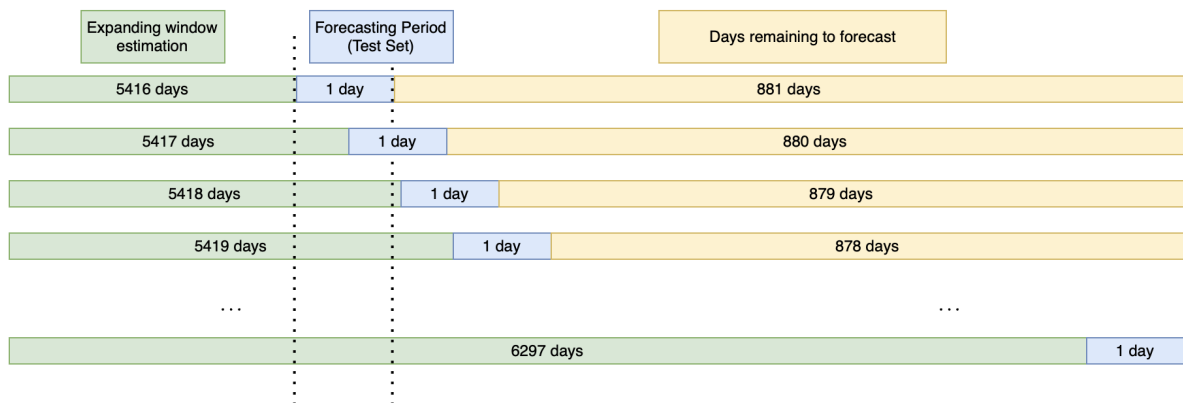


Figure 4.5: Expanding window scheme used for the GARCH model forecasts.

4.6 Forecasting using Neural Networks

4.6.1 Proposed Hybrid Models

The hybrid models used in this experiment use the forecasts from the GARCH(1,1) with a 252 day rolling window as an additional feature within the deep feed-forward (DFN)¹ and LSTM networks [21] [39], are denoted as the GARCH-DFN and GARCH-LSTM models. However, an important caveat is the loss of 252 training observations in order to generate the first forecast used as an input into the neural networks. Table 4.4, shows the updated training, validation and test split. The validation and test split remain unchanged but the training data is reduced. All neural networks will be trained on the reduced training data.

Data Split	Observations	% of Updated Dataset
Training (Reduced)	4282	70.82%
Validation	882	14.59%
Test	882	14.59%
Total (Updated)	6046	100%

Table 4.4: Updated dataset split after reducing the initial training size (original: 4534).

4.6.2 Feature Engineering and Data Preprocessing

To evaluate the incremental value of different features, three neural network models are constructed using a progressively larger set of features.

1. Squared Log Returns (DFN and LSTM).
2. Squared Log Returns and the GARCH Forecast (GARCH-DFN and GARCH-LSTM).
3. Squared Log Returns, GARCH Forecast, and Log Returns (LR-GARCH-DFN and LR-GARCH-LSTM).

The squared log returns are used on the basis that the GARCH model is not able to fully capture the behaviour of the squared returns, particularly in volatile periods [39]. The log returns are included to attempt to capture the asymmetric impact of volatility. The sequential addition of features ensures that any change in forecasting accuracy can be solely attributed to the additional feature. Table 4.5 summarises the inputs used for each model.

	DFN	LSTM	GARCH-DFN	GARCH-LSTM	LR-GARCH-DFN	LR-GARCH-LSTM
Squared Log Returns	O	O	O	O	O	O
GARCH(1,1) Forecasts	X	X	O	O	O	O
Log Returns	X	X	X	X	O	O

Note: The 'O' denotes that the variable was used, the 'X' that the variable was not used in the model.

Table 4.5: Input features used for each model.

¹A deep feed-forward network is simply a feedforward network with more than one hidden layer.

Data Normalisation

Data normalisation is an important part of the preprocessing of data before fitting a neural network. The MinMax scaler works to scale each feature within a given range. For this experiment, the feature range is set between $(0, 1)$. For an input vector, $\mathbf{x}$, the MinMax scaler is defined as

$$x'_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}, \quad (4.8)$$

where x'_i is the scaled value corresponding to the original value $x_i \in \mathbf{x}$ and $x_{\min}$ and $x_{\max}$ are the minimum and maximum values in $\mathbf{x}$, respectively.

The scaler was applied to each feature within the data after the dataset was split into the train, validation and test splits. This is done to minimise data leakage. If the entire dataset was scaled together, then the training and validation sets would be scaled using information from future time steps.

Windowed Format

The number of previous observations used as input into the neural network prior to making a prediction is known as the *lookback window*. A lookback window of 21 days is used in this experiment, with a one-day forecast horizon, meaning the model uses the previous 21 days to predict one day ahead. This window size is chosen as it is the approximate number of trading days in a month. Unlike LSTMs, as feed-forward networks cannot inherently handle sequential data, each input node will correspond to one day of data. For m features, the number of input nodes are $m \times 21$.

4.6.3 Model Selection and Validation

There are several considerations which can significantly affect the network's in and out-of-sample performance.

Batch Size and Loss Functions

The choice of *batch size* and loss function are two important considerations when forecasting with neural networks. To ensure comparable results, these two choices remain consistent throughout the experiment.

The *batch size* refers to the number of samples processed in one forward and backward pass. For Equation (2.40), this corresponds to the number of samples, N , in one batch before updating the parameters. By using a batch-size less than the number of total observations, the memory requirements for training the network are reduced. In general, smaller batch sizes are preferred as the models tend to generalise better to unseen data [40]. However, excessively small batch sizes can lead to high gradient variance, potentially hindering convergence to a local minima.

For this paper, the MAE, defined in Equation (4.2), is chosen as the loss function across all neural networks and a batch size of 128 is used. Appendix A.2 details the reasoning behind this choice.

Regularisation

Overfitting is a common issue in neural networks which stems from their ability to model incredibly complex and non-linear relationships. Overfitting occurs when the network

learns the training data too precisely, and is characterised by a low training loss but a significantly higher validation loss, indicating the network has failed to generalise to unseen data. Two regularisation techniques are employed to prevent overfitting.

The first technique used to prevent overfitting is known as early stopping. If the validation loss has not improved after a specified number of epochs (iterations over the training data), early stopping will stop the gradient descent algorithm prematurely [27]. The number of epochs until the gradient descent algorithm is stopped early is known as the patience.

For this experiment, early stopping is employed on the validation loss with a patience of 75 epochs, giving the network time to improve while avoiding overfitting once the loss plateaus. The maximum number of epochs is set to 200 to bound computational cost and ensure the model stops training even if early stopping is not triggered.

The second technique employed is known as *dropout*. Dropout layers aim to make the network more robust by temporarily removing certain hidden units from the network during each forward pass, making units less dependent on one another [41]. In practise, dropout layers are implemented before a layer of fully-connected neurons (fully connected layer) using inverted dropout. With inverted dropout implemented, Equation (2.33) becomes

$$r_j^{(l)} \sim \text{Bernoulli}(p) \quad (4.9)$$

$$\tilde{\mathbf{A}}^{(l)} = \frac{1}{p} \mathbf{r}^{(l)} \odot \mathbf{A}^{(l)} \quad (4.10)$$

$$\mathbf{Z}^{(l+1)} = \mathbf{W}^{(l+1)} \tilde{\mathbf{A}}^{(l)} + \mathbf{b}^{(l+1)} \quad (4.11)$$

$$\mathbf{A}^{(l+1)} = g^{(l+1)}(\mathbf{Z}^{(l+1)}) \quad (4.12)$$

where $\mathbf{r}^{(l)} \in \{0, 1\}^j$ is the vector of independent Bernoulli random variables with $\mathbb{P}(\mathbf{r}_j^{(l)} = 1) = p$ for the l^{th} layer with j units. [41].

Hyperparameter Tuning

Finding appropriate model parameters and hyperparameters is a crucial aspect of building a robust model. The model parameters include the number of hidden layers, the number of hidden units per layer and the probability of dropout applied before each fully connected layer. The hyperparameters in this experiment are the learning rate and the number of epochs. The model which yields the lowest HMSE over the validation set is selected as the preferred model architecture [14].

Initial model configurations for the DFN models were taken from Kim and Won where the number of layers, hidden size and dropout were then fine-tuned through trial and error to obtain the model configurations that yielded the lowest forecast error [17]. All the DFN models had between 2-3 layers and between 4-16 units per layer. As the input size increased, the number of layers and units per layer increased to capture the additional complexity. The dropout remained the same throughout with a dropout of 0.2 and 0.1 before the first and second layers, respectively.

The initial model configurations for the LSTM models were taken from a Kakade et al. and then fine-tuned to obtain the model with the lowest forecast error [21]. All LSTM models had one LSTM layer, and the hybrid LSTM models had an additional fully connected layer after the LSTM layer with a dropout to capture the additional model complexity without overfitting.

The optimal number of epochs was found by recording the epoch number when the lowest validation loss (MAE) is achieved. This is the number of epochs used for final testing of the model [27]. The Adam optimiser [31] with a learning rate of 0.001 was used for all models.

Table 4.6 summarises the architecture of each model and the optimal hyperparameters. Table 4.7 shows the performance of the models on the validation set, this result represents the lowest HMSE that could be achieved through the trial and error approach.

	DFN	LSTM	GARCH-DFN	GARCH-LSTM	LR-GARCH-DFN	LR-GARCH-LSTM
Number of layers	2	1	3	2	3	2
Hidden units/dimension per layer	14, 8	30	16, 8, 4	40 + 16 FC ¹	16, 10, 4	40 + 16 FC ¹
Dropout	0.2, 0.1 ²	0.0	0.2, 0.1 ²	0.2	0.2, 0.1 ²	0.2
Learning rate	0.001	0.001	0.001	0.001	0.001	0.001
Number of epochs	167	98	31	106	171	6

Table 4.6: Model architecture and training parameters for each model configuration.

¹ “FC” denotes a fully connected layer appended after the LSTM layer.

² For DFN-based models, dropout values are listed in order of the layer they precede.

Model	MAE	HMAE	MSE	HMSE
DFN	0.004109	0.3132	6.509e-05	0.1432
LSTM	0.004103	0.3080	6.180e-05	0.1417
GARCH-DFN	0.003987	0.3035	6.485e-05	0.1341
GARCH-LSTM	0.003998	0.3049	5.984e-05	0.1407
LR-GARCH-DFN	0.003908	0.3018	6.004e-05	0.1324
LR-GARCH-LSTM	0.003959	0.3004	6.448e-05	0.1364

Table 4.7: Error metrics for all models over the validation set.

4.6.4 Out-of-Sample Forecasting

Once the models are trained on the reduced training data, and model parameters and hyperparameters are confirmed using the validation data, the model is then retrained on the combined training and validation data and forecast over the test set, as outlined in Table 4.4. Figure 4.6 summarises the model selection and forecasting process.

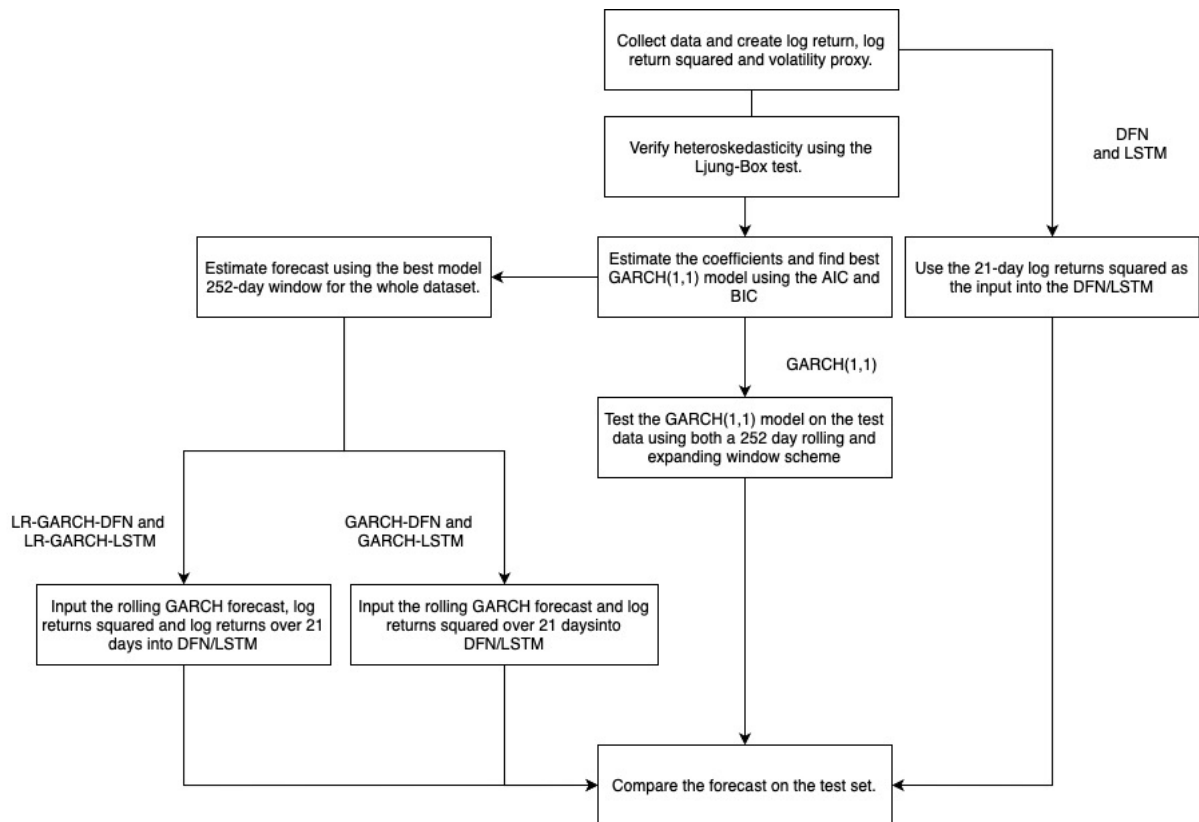


Figure 4.6: Overview of model selection and forecast process.

Chapter 5

Results and Discussion

5.1 Results

Nine models were created, and forecasts were generated over the entire test set. The models were evaluated using the MAE, MSE, HMAE and HMSE. Table 5.1 summarises the out-of-sample forecast errors of the individual GARCH, DFN and LSTM models along with the respective hybrid and extended hybrid models.

Summarised by Figures 5.3 and 5.4, both the rolling and expanding window GARCH(1,1) models outperformed the benchmark model with respect to all metrics. The expanding window variant performed the best overall with respect to all metrics except the HMSE. The expanding window model achieved a 3.14% and 2.02% decrease from the benchmark, in HMAE and HMSE, respectively. Moreover, across all models, the expanding window showed the greatest improvement compared to the benchmark in MAE and MSE, with a reduction of 7.76% and 17.73% respectively. Although the expanding window model outperformed the rolling window model by 5.44% and 8.96% in terms of MAE and MSE, respectively, the rolling window model outperformed the expanding window by 4.74% in terms of HMSE, representing a 6.66% improvement from the benchmark. The forecasts of the GARCH models for the two window schemes is shown in Figure 5.1.

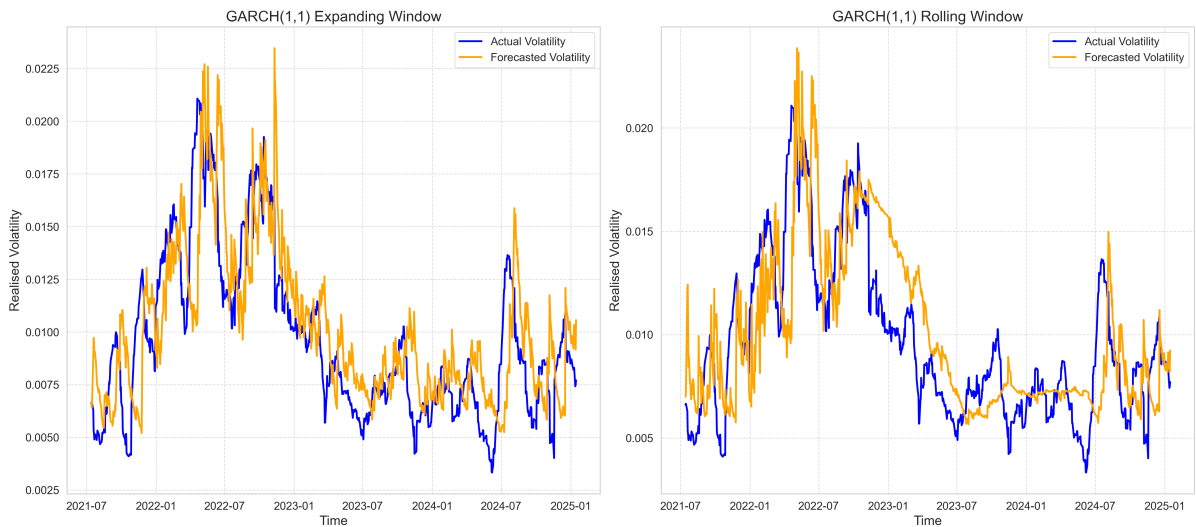


Figure 5.1: Forecasts from the GARCH(1,1) model with expanding window (Left) and the rolling window (Right).

Figure 5.3 and 5.4 show the DFN performed poorly, failing to outperform the benchmark in all metrics except the HMSE. Specifically, it exhibited an increase in HMAE by 6.78% compared to the benchmark. However, relative to the benchmark, the DFN did yield a reduction in HMSE by 14.69%, and significantly outperformed both the GARCH(1,1) models with respect to this metric, achieving a 8.60% reduction in HMSE from the GARCH(1,1) with a rolling window. The DFN forecast over the test set is shown in Figure 5.2.

As illustrated in Figures 5.3 and 5.4, the LSTM outperformed the DFN in all metrics, and outperformed the benchmark in terms of HMAE and HMSE. It achieved a 9.31% and 24.77% improvement in HMAE and HMSE from the benchmark, respectively. When compared to the DFN, the model had a 15.07% reduction in HMAE and an 11.82% reduction in HMSE. Additionally, the LSTM outperformed both GARCH(1,1) models, showing a 6.37% reduction in HMAE from the GARCH(1,1) with an expanding window and a 19.40% reduction in HMSE compared to the GARCH(1,1) with a rolling window. The LSTM forecast can be seen in Figure 5.2.

For all metrics, the GARCH-DFN and GARCH-LSTM outperformed their corresponding stand-alone neural networks. Summarised by Figure 5.5, DFN benefited the most from the addition of GARCH forecast. The GARCH-DFN achieved a significant improvement, reducing the HMAE by 17.32% and the HMSE by 21.74%, from the DFN. The GARCH-LSTM also saw an improvement from the LSTM with a reduction of 8.49% and 16.62% for the HMAE and HMSE, respectively.

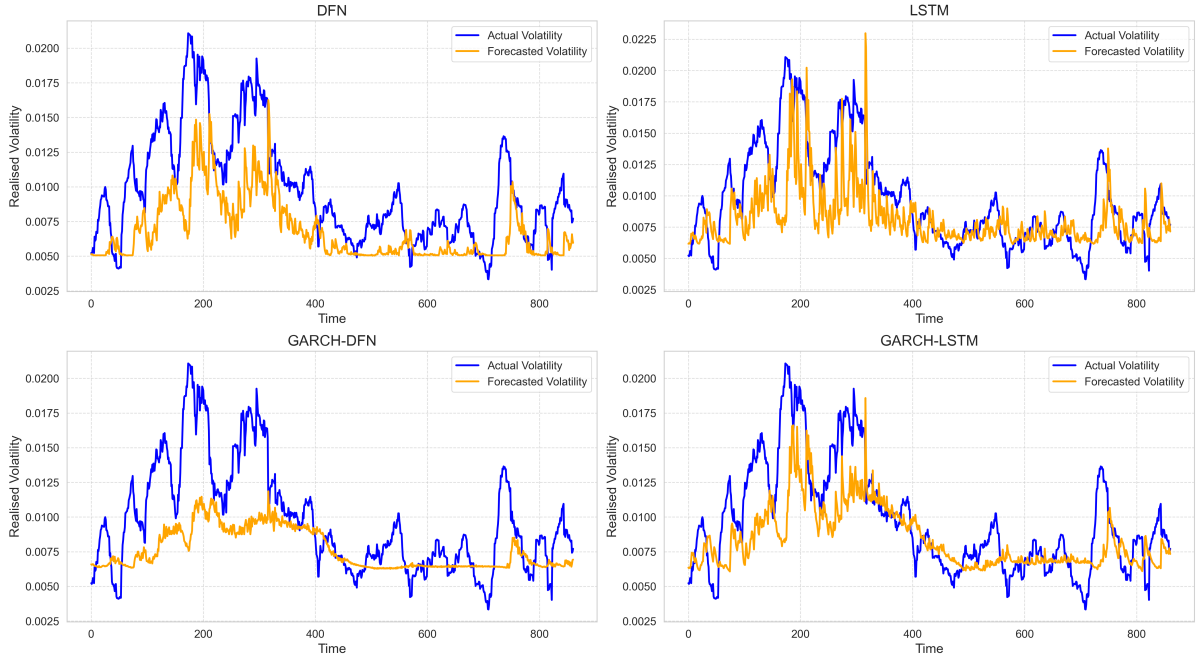


Figure 5.2: Forecasts from the DFN (Top Left), LSTM (Top Right), GARCH-DFN (Bottom Left) and GARCH-LSTM (Bottom Right) model.

Figure 5.3 further shows that both hybrid models offer a significant improvement in terms of HMAE and HMSE over the GARCH(1,1) models. Notably, the GARCH-LSTM offers a 14.33% improvement in HMAE compared to the GARCH(1,1) with an expanding window and a 32.80% improvement in HMSE compared to the GARCH(1,1) with a rolling window. Table 5.2 summarises the reduction in HMAE and HMSE achieved

by the GARCH-LSTM, relative to the other models.

However, Figure 5.4 highlights that both hybrid models still struggled to outperform the GARCH(1,1) with an expanding window in terms of MAE and MSE. While the GARCH-LSTM marginally outperformed the GARCH(1,1) with a rolling window in terms of MAE, the expanding window variant still offered a 4.98% and 22.94% reduction from the GARCH-LSTM in MAE and MSE, respectively.

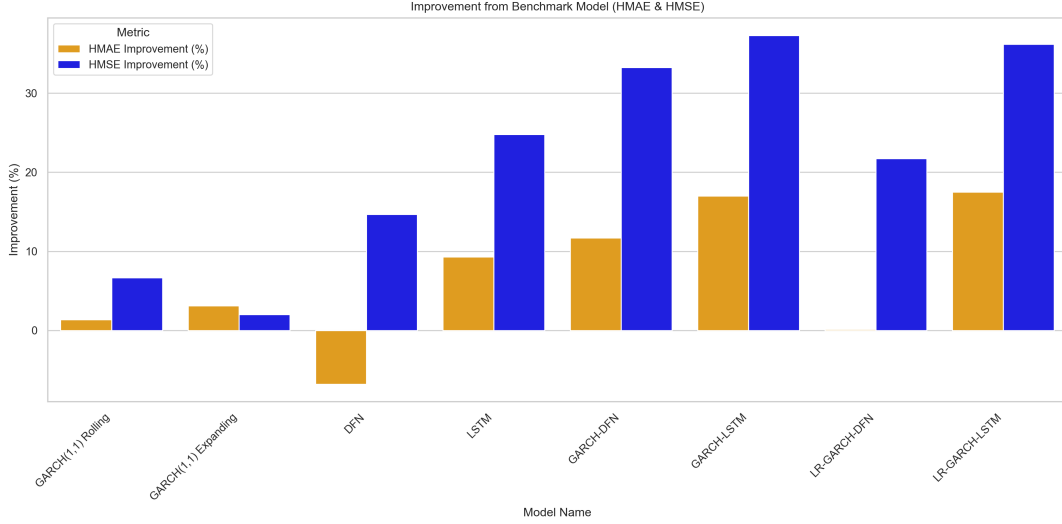


Figure 5.3: Improvement (%) in terms of HMAE and HMSE from the benchmark.

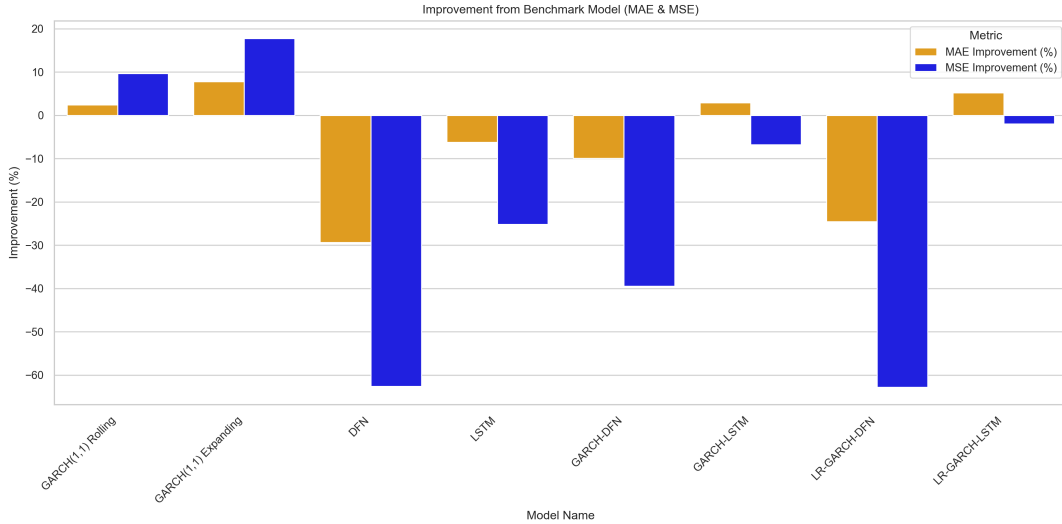


Figure 5.4: Improvement (%) in terms of MAE and MSE from the benchmark.

Figure 5.3 and Figure 5.4 show that the addition of the log returns into the hybrid model yielded mixed results. The additional feature hindered the GARCH-DFN, where the HMAE and HMSE were increased by 13.09% and 17.26%, respectively. While, the LR-GARCH-LSTM did see improvements across the MAE, MSE and HMAE, the difference was marginal, with only a 0.59% improvement in HMAE. The LR-GARCH-LSTM did show superior performance compared to the GARCH(1,1) with a rolling window, reducing

the MAE by 2.82% but failed to outperform the expanding window variant in terms of MAE or MSE. The forecasts for both extended hybrid models are shown in Figure 5.2.

To summarise, in terms of MAE and MSE, the GARCH(1,1) with an expanding window performed the best when forecasting over the test set, showing a 4.98% and 22.94% reduction in MAE and MSE when compared to the GARCH-LSTM. The DFN performed poorly against the benchmark, and only outperformed the GARCH models in terms of HMSE, however the model saw improvements with the introduction of the GARCH forecast. With respect to HMAE, the hybrid GARCH-LSTM offered a 8.49% reduction and a 16.62% reduction in HMSE from the LSTM. The extended LR-GARCH-LSTM yielded mixed results, and ultimately did not show any significant outperformance compared to the GARCH-LSTM.

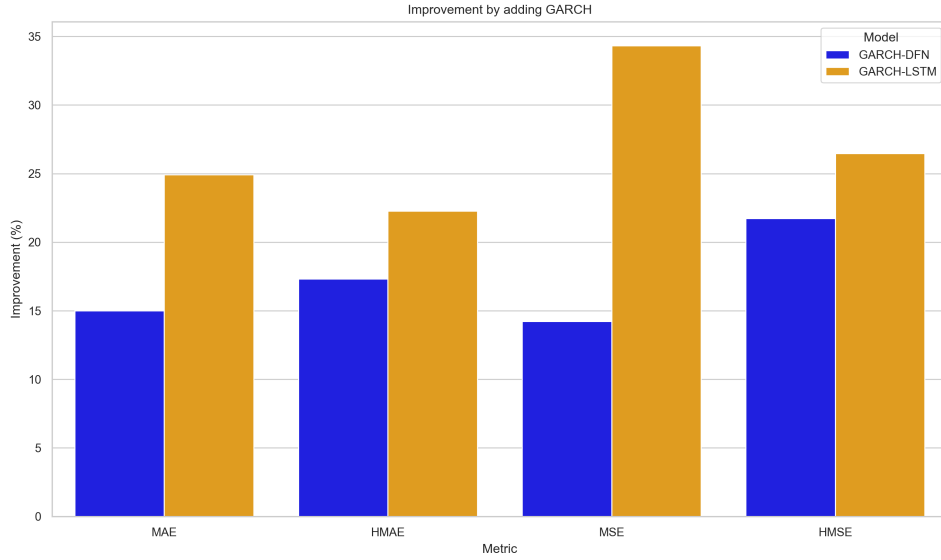


Figure 5.5: Improvement (%) to the DFN and LSTM models in terms of all metrics when the GARCH forecasts are added as an input.

Model Name	MAE	HMAE	MSE	HMSE
Benchmark Model	0.002447	0.2731	1.026e-05	0.1283
GARCH(1,1) Rolling	0.002388	0.2693	9.268e-06	0.1197
GARCH(1,1) Expanding	0.002258	0.2645	8.437e-06	0.1257
DFN	0.003165	0.2916	1.667e-05	0.1094
LSTM	0.002601	0.2477	1.284e-05	0.09649
GARCH-DFN	0.002690	0.2411	1.430e-05	0.08564
GARCH-LSTM	0.002376	0.2266	1.095e-05	0.08046
LR-GARCH-DFN	0.003047	0.2727	1.669e-05	0.1004
LR-GARCH-LSTM	0.002320	0.2253	1.046e-05	0.08187

Note: Values in bold indicate the lowest forecast errors for each metric.

Table 5.1: Performance results when forecasting on the test set.

Model	HMAE (%)	HMSE (%)
Benchmark Model	-17.01	-37.27
GARCH(1,1) Rolling	-15.86	-32.80
GARCH(1,1) Expanding	-14.33	-35.98
DFN	-22.28	-26.47
LSTM	-8.49	-16.62
GARCH-DFN	-6.00	-6.05
GARCH-LSTM	0.00	0.00
LR-GARCH-DFN	-16.88	-19.88
LR-GARCH-LSTM	0.59	-1.72

Table 5.2: Reductions (%) in HMAE and HMSE from the GARCH-LSTM relative to other models.

5.2 Discussion

The findings of this experiment are consistent with those of earlier work, demonstrating that the LSTM can offer significant improvements over the GARCH in terms of both HMAE and HMSE [17] [19]. This outperformance is particularly notable when using the expanding window scheme for the GARCH model, where the model is re-fit on all data up to the forecast point. In contrast, the LSTM was not updated during test, and only had a lookback window of 21 days, yet it still outperformed the GARCH model, suggesting its superior ability to learn and generalise temporal patterns in financial time series. This indicates that the LSTM can more effectively generalise to unseen data as opposed to the GARCH model, and that given a longer lookback window, this outperformance may have been even more significant.

In contrast to Hu et al., who reported that both the DFN and GARCH-DFN models, with four to six hidden layers, consistently outperformed the GARCH model across all metrics, this experiment found that neither the DFN nor the GARCH-DFN models outperformed even the benchmark, except in terms of HMSE [19]. This discrepancy potentially highlights the need for further model complexity, as employed by Hu et al., or a network designed to handle temporal dependencies, such as the LSTM.

Once again, in accordance with previous research, this experiment established improvements across all metrics when incorporating the GARCH forecasts into the DFN and LSTM models, confirming the benefits of the hybrid neural network as opposed to the stand-alone ANNs [21] [19]. Specifically, these findings are in accordance with Kim and Won, who similarly observed a reduction in forecast error across all metrics when GARCH forecasts were integrated into DFN and LSTM models [17]. Such consistency of results suggests that incorporating the conditional volatility serves as a useful inductive bias for the two neural networks when modelling the volatility of financial time series. Nonetheless, while the LSTM and all hybrid models perform well, they also significantly increase model complexity with regards to hyperparameter tuning and overall training time, relative to the GARCH model. This can cause a greater variability in outcomes and higher computational costs when developing state-of-the-art forecasting models.

The inclusion of the log returns did not significantly improve either of the models. With the additional feature added, the performance of the GARCH-DFN notably deteriorated, likely due to the introduction of additional noise and exacerbated by limited

model complexity. Although the GARCH-LSTM was more robust to the additional feature, it demonstrated no clear improvement, particularly in terms of HMAE and HMSE. Given that the asymmetric impact of returns is a well-documented phenomenon within financial time series, these results suggest that a feature which captures this asymmetry more effectively, may be required.

There is a discrepancy between the performance of the models with respect to scale-variant and scale-invariant metrics. Concerning the scale-invariant metrics, namely the MAE and MSE, the GARCH models performed the best overall, with even the benchmark model outperforming all neural networks in terms of MSE. This lower forecasting error is likely attributable to the GARCH models' stronger during lower volatility periods, particularly towards the end of the test set. A similar observation was also noted by Krisjanpoller and Minutolo, who, in their study of the volatility of the gold spot and future price found that while the MAPE improved with hybrid GARCH model, the GARCH model continued to have the lowest MAE and MSE across all approaches [13].

On the other hand, the LSTM and the hybrid GARCH models performed considerably better than the GARCH models when evaluated using scale-variant error metrics, namely the HMSE and HMAE. This may indicate that neural networks, particularly the LSTM and GARCH-LSTM, are more adept at handling high volatility regimes. In contrast, all the neural networks struggled to adapt to the lower volatility regime, compared to the GARCH model. This observation is consistent with the findings of Monfared and Enke, who reported that the hybrid model underperformed a GARCH approach during more stable periods [22]. Taken together, this experiment highlights the trade-off between adaptability and stability. While the LSTM and hybrid models perform well in high volatility periods and under scale-variant metrics, traditional approaches such as the GARCH, retain an advantage during more stable market conditions, especially when assessed using scale-invariant metrics.

Finally, for the GARCH models, the expanding window outperformed the rolling window approach, except in terms of HMSE. Figure 5.1 illustrates this, showing it is more effective at adjusting to regime shifts, particularly from the high to low volatility period towards the end of the test set. This is likely due to the fact that the expanding window continually incorporates all past data into its forecast, enabling it to better model changes in regime. In contrast, the rolling window model only considers a fixed recent subset of data, making it slower to adapt to regime changes when that subset lacks a similar pattern. Interestingly, the rolling window GARCH model performed slightly better in terms of HMSE, suggesting its superiority during periods of constant high volatility. Ultimately, while the GARCH model with a rolling window may offer advantages during constant high volatility regimes, the expanding window demonstrates broader robustness for all regimes, and excels during regime changes.

Chapter 6

Conclusion

6.1 Conclusions

This thesis sought to investigate whether hybrid GARCH-ANNs can significantly improve the forecasting accuracy of ANNs and traditional GARCH models, particularly during low volatility periods.

To do so, a theoretical background on time series analysis, autoregressive conditional heteroscedasticity models and neural networks, was first provided to situate this research and motivate the experiment. Subsequently, combining previous methodology from recent literature, the design of the experiment and how it aimed to answer the central question of this thesis was detailed. All the models were fit over a subsection of the data and then forecasted on an out-of-sample period and evaluated against the 21-day realised volatility measure.

From the results of the experiment, three central conclusions can be established. First, the findings of this thesis confirm the majority of research, which suggests that the hybrid GARCH-DFN and GARCH-LSTM outperform the stand-alone DFN and LSTM models. The benefit of utilising a hybrid model for volatility forecasting is therefore highlighted. Accordingly, such findings suggest that a hybrid GARCH-LSTM model is the most suitable for those looking to minimise the relative error of their volatility forecasts, such as investors.

Secondly, the findings of this thesis are consistent with past research on hybrid GARCH-ANN models. Whilst the hybrid models outperformed the GARCH model in terms of HMAE and HMSE on the low volatility test data, most of the neural networks struggled to outperform the GARCH model in terms of MSE and MAE. This indicates that although the hybrid models can outperform the GARCH model relative to the current volatility regime, the neural networks struggle to outperform the GARCH model on an absolute scale. Taking into account the relatively low variance of the test set, this could indicate that compared to ANNs, simpler models (such as the GARCH) should be preferred for volatility forecasting during lower volatility periods. Moreover, such findings suggest that a GARCH model may be more useful, in comparison to ANNs, for applications that value the minimisation of the absolute loss over the relative one. On the contrary, the LSTM models in this experiment, may be more useful for forecasting during high volatility periods.

Lastly, this thesis finds that while the asymmetric impact of returns on volatility were present within the data, the addition of the log returns into the hybrid model did not result in any significant improvements to the GARCH-LSTM, especially with regards to

the HMSE and HMAE. Therefore, the findings of this thesis draws attention to the need for a feature which better targets this asymmetric effect.

6.2 Future Work

Looking forward, there are a multitude of areas in which this model for volatility could be furthered. While the GARCH model performed well during low volatility periods, it appeared to struggle in higher volatility periods. This could be due to the asymmetric impact of returns on volatility, wherein the negative returns have a higher impact on volatility. In future work, various extensions of the GARCH model which capture this asymmetric impact, such as the EGARCH, could be considered [8]. Numerous studies have confirmed the EGARCH's superior performance over the GARCH model [15] [17]. Moreover, due to mixed results with incorporating the log returns to capture this behaviour in the ANNs, a more suitable model could be a hybrid EGARCH-ANN or EGARCH-GARCH-ANN [18] [20]. Additionally, as briefly mentioned the distribution of the standardised residuals is typically assumed to be Gaussian, however certain papers have indicated that distributions, such as the Student-t distribution, may be more suitable [10].

As the window sizes and forecasting horizon for the GARCH and ANN models remained fixed throughout this paper, a potential direction for future research could entail the investigation of increasing or decreasing of these parameters. For the GARCH model, another popular choice of rolling window is the 504-day window, representing two years of trading, which may be better equipped to balance the responsiveness of a shorter window with the stability of the expanding window. Moreover, as the rolling window is updated at every daily time step, this could have resulted in noisier forecasts. Using a larger window size in combination with a window that updates every few months may yield a more robust model [7]. Additionally, the look-back window of the ANNs could be altered to investigate if they can further outperform the GARCH model with a slightly larger lookback window. While for both models, the forecast horizon was only one day ahead, many applications in finance, such as option pricing, may be concerned with the price of the asset after a longer period. As such, more research is needed into how these models perform under varying forecast horizons.

Moreover, further research could introduce intra-day data to provide a more representative ex-post volatility measure, in addition to improving model performance. As shown by both Anderson and Bollerslev, in addition to, Macmillian and Speight, the reported inability of the GARCH models to significantly outperform simple statistical and moving average models can be rejected if a measure, such as the 5-minute realised variance, is used as the ex-post volatility measure [35][42]. Utilising high-frequency data provides a less noisy measure compared to squared returns, and a more representative measure than the rolling 21-day realised volatility. Additionally, over the past 20 years, there has been an increasing amount of research into integrating these high-frequency realised volatility measures into volatility models, such as the Realised-GARCH model [43]. These models, especially when combined with LSTMs, have demonstrated significant improvement in performance when forecasting volatility in comparison to the traditional GARCH models [44].

Lastly, as the log returns did not add any significant improvement to the hybrid model, alternative explanatory variables could be investigated in future research. Volatility in-

dexes, such as the VIX, which uses S&P 500 option prices to determine market uncertainty, could be a beneficial feature to capture market sentiment. Additionally, promising research from Xiong et al. showed that including Google domestic trends into an LSTM significantly improved the performance of the LSTM over traditional econometric models [45].

Appendix A

Appendix

A.1 Choice of Window Size for Volatility Proxy

Figure A.1 demonstrates how the choice of window size for Equation (4.1) impacts the proxy, and how it compares with the absolute log returns. It is clear that as the window size decreases, the respective proxies become noisier, with the absolute log returns exhibiting the highest variance of all proxies. It has long been known that the log returns squared (and the absolute log returns) are a noisy (and poor) proxy for volatility [34], and when used, often results in apparent poor performance of GARCH type models [35] [42]. While a shorter rolling window size results in a noisier proxy, proxies with a longer window are less responsive to recent market changes. The 21-day window balances the responsiveness of a proxy to recent market events with the robustness required for model training and evaluation.

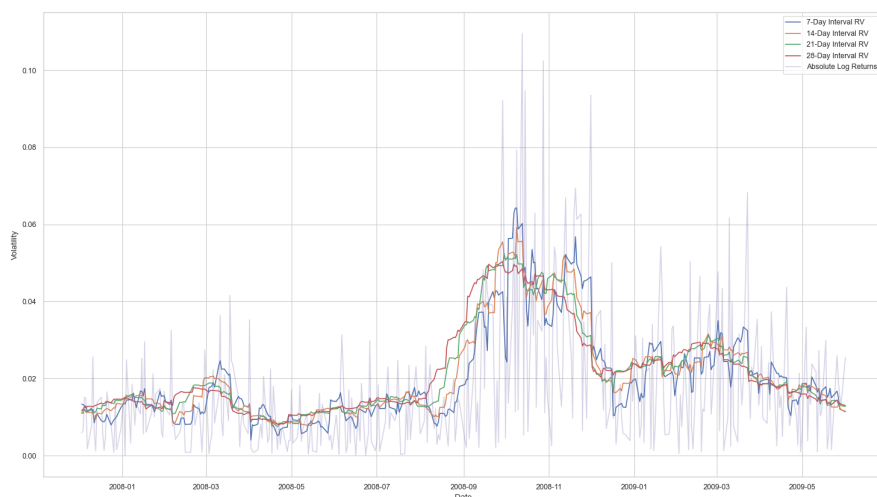


Figure A.1: Volatility proxies over the highly volatile period of 2008-09. The proxies compared are the absolute log returns and the realised volatility (Equation 4.1) with 7, 14, 21, 28 day windows.

A.2 Choice of Loss Function and Batch Size

During preliminary research, the mean squared error (MSE),

$$\mathcal{L}(\boldsymbol{\theta}) := \frac{1}{N} \sum_{i=1}^N \|f(\mathbf{x}^{(i)}; \boldsymbol{\theta}) - \mathbf{y}^{(i)}\|^2, \quad (\text{A.1})$$

was also considered as a loss function. Compared to the MAE, the MSE penalises large errors more heavily. However, for data with a large amount of outliers, this sensitivity could impact the consistency in the model’s performance.

During preliminary research, an initial experiment was conducted investigating the effect of the batch size and chosen loss function on an MLP’s performance on the validation set. The initial experiment sought to select an optimal batch size and loss function which not only obtained the best HMAE and HMSE on the validation set, but also had a low standard deviation across different initial parameter choices to choose a model which is less sensitive to its starting values.

The initial model, based off of Kim and Won, was a DFN with two hidden layers with 15 and 10 units and no dropout [17]. The model was trained with 10 randomly generated initial parameter values each time to assess sensitivity to parameters. The batch sizes evaluated was 32, 64 and 128. Two different learning rates of 0.001 and 0.0001 were trialled for each configuration. Table A.1 and Table A.2 detail the results.

For a learning rate of 0.001, the MAE with a batch size of 128 obtained the best results, in terms of both minimising the average forecast errors and the standard deviation of the errors. This was also found for a smaller learning rate of 0.0001, however, the MAE with a smaller batch size of 32 was preferred in terms of minimising the HMSE but does not minimise the variance of the error. As such, the MAE with a batch size of 128 was chosen.

Table A.1: Model Performance Metrics (Learning Rate = 0.001)

Loss Function	Batch Size	Average HMAE	StdDev HMAE	Average HMSE	StdDev HMSE
MAE	128	0.3628	0.0271	0.1942	0.0350
MAE	64	0.3914	0.0358	0.2256	0.0534
MAE	32	0.4157	0.0922	0.2338	0.0792
MSE	128	0.4771	0.1026	0.3915	0.1827
MSE	64	0.3979	0.0894	0.2602	0.1477
MSE	32	0.4118	0.1038	0.2828	0.1799

Table A.2: Model Performance Metrics (Learning Rate = 0.0001)

Loss Function	Batch Size	Average HMAE	StdDev HMAE	Average HMSE	StdDev HMSE
MAE	128	0.3770	0.0263	0.2197	0.0423
MAE	64	0.3805	0.0327	0.2179	0.0454
MAE	32	0.3876	0.0728	0.2150	0.0634
MSE	128	0.4826	0.0871	0.4013	0.1573
MSE	64	0.4518	0.1250	0.3604	0.2378
MSE	32	0.4332	0.0974	0.3170	0.1674

Bibliography

- [1] Poon SH, Granger CWJ. Forecasting Volatility in Financial Markets: A Review. *Journal of Economic Literature*. 2003 June;41(2):478-539. Available from: <https://ideas.repec.org/a/aea/jecclit/v41y2003i2p478-539.html>.
- [2] Black F, Scholes M. The Pricing of Options and Corporate Liabilities. *Journal of Political Economy*. 1973;81(3):637-654. Available from: <https://www.jstor.org/stable/1831029>.
- [3] Mandelbrot B. The Variation of Certain Speculative Prices. *The Journal of Business*. 1963 Jan;36(4):394. Available from: https://web.williams.edu/Mathematics/sjmiller/public_html/341Fa09/econ/Mandelbroit_VariationCertainSpeculativePrices.pdf.
- [4] Engle RF, Patton AJ. What good is a volatility model? *Quantitative Finance*. 2001 Feb;1(2):237-245. Available from: <https://web-static.stern.nyu.edu/rengle/EnglePattonQF.pdf>.
- [5] Bollerslev T. Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*. 1986 Apr;31(3):307-327.
- [6] Akgiray V. Conditional Heteroscedasticity in Time Series of Stock Returns: Evidence and Forecasts. *The Journal of Business*. 1989;62(1):55-80. Available from: <https://www.jstor.org/stable/2353123>.
- [7] Figlewski S. Forecasting Volatility. *Financial Markets, Institutions and Instruments*. 1997 Feb;6(1):1-88. Available from: <https://onlinelibrary.wiley.com/doi/epdf/10.1111/1468-0416.00009>.
- [8] Nelson DB. Conditional Heteroskedasticity in Asset Returns: a New Approach. *Econometrica*. 1991 Mar;59(2):347-370. Available from: <https://www.jstor.org/stable/2938260>.
- [9] Nelson DB, Cao CQ. Inequality Constraints in the Univariate GARCH Model. *Journal of Business Economic Statistics*. 1992 Apr;10(2):229. Available from: <https://www.jstor.org/stable/1391681?seq=1>.
- [10] Wilhelmsson A. Garch forecasting performance under different distribution assumptions. *Journal of Forecasting*. 2006;25(8):561-578. Available from: <https://onlinelibrary.wiley.com/doi/epdf/10.1002/for.1009>.

- [11] Christie AA. The stochastic behavior of common stock variances: Value, leverage and interest rate effects. *Journal of Financial Economics*. 1982;10(4):407-32. Available from: <https://www.sciencedirect.com/science/article/pii/0304405X82900186>.
- [12] Donaldson RG, Kamstra M. An artificial neural network-GARCH model for international stock return volatility. *Journal of Empirical Finance*. 1997 Jan;4(1):17-46. Available from: [https://doi.org/10.1016/S0927-5398\(96\)00011-4](https://doi.org/10.1016/S0927-5398(96)00011-4).
- [13] Kristjanpoller W, Minutolo MC. Gold price volatility: A forecasting approach using the Artificial Neural Network-GARCH model. *Expert Systems with Applications*. 2015;42(20):7245-51. Available from: <https://www.sciencedirect.com/science/article/pii/S0957417415003000>.
- [14] Kristjanpoller W, Minutolo MC. Forecasting volatility of oil price using an artificial neural network-GARCH model. *Expert Systems with Applications*. 2016 Dec;65:233-241. Available from: <https://www.sciencedirect.com/science/article/pii/S0957417416304420>.
- [15] Hajizadeh E, Seifi A, Fazel Zarandi MH, Turksen IB. A hybrid modeling approach for forecasting the volatility of SP 500 index return. *Expert Systems with Applications*. 2012;39(1):431-6. Available from: <https://www.sciencedirect.com/science/article/pii/S095741741100995X>.
- [16] Hochreiter S, Schmidhuber J. Long Short-Term Memory. *Neural Computation*. 1997 11;9(8):1735-80. Available from: <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [17] Kim HY, Won CH. Forecasting the volatility of stock price index: A hybrid model integrating LSTM with multiple GARCH-type models. *Expert Systems with Applications*. 2018 Aug;103:25-37. Available from: <https://www.sciencedirect.com/science/article/pii/S0957417418301416>.
- [18] Hyup Roh T. Forecasting the volatility of stock price index. *Expert Systems with Applications*. 2007;33(4):916-22. Available from: <https://www.sciencedirect.com/science/article/pii/S0957417406002223>.
- [19] Hu Y, Ni J, Wen L. A hybrid deep learning approach by integrating LSTM-ANN networks with GARCH model for copper price volatility prediction. *Physica A: Statistical Mechanics and its Applications*. 2020;557:124907. Available from: <https://www.sciencedirect.com/science/article/pii/S0378437120304696>.
- [20] Kartsonakis Mademlis D, Dritsakis N. Volatility Forecasting using Hybrid GARCH Neural Network Models: The Case of the Italian Stock Market. *International Journal of Economics and Financial Issues*. 2021 Jan;11(1):49-60. Available from: <https://econjournals.com/index.php/ijefi/article/view/10842>.
- [21] Kakade K, Mishra AK, Ghate K, Gupta S. Forecasting Commodity Market Returns Volatility: A Hybrid Ensemble Learning GARCH-LSTM based Approach. *Intelligent Systems in Accounting, Finance and Management*. 2022;29(2):103-17. Available from: <https://onlinelibrary.wiley.com/doi/abs/10.1002/isaf.1515>.

- [22] Monfared SA, Enke D. Volatility Forecasting Using a Hybrid GJR-GARCH Neural Network Model. *Procedia Computer Science*. 2014;36:246–253. Available from: <https://doi.org/10.1016/j.procs.2014.09.087>.
- [23] Kirchgässner G, Wolters J. *Introduction to Modern Time Series Analysis*. 2nd ed. Europapl. 3, 69115 Heidelberg, Germany: Springer; 2012. Available from: <https://link.springer.com/book/10.1007/978-3-642-33436-8>.
- [24] NIST. *e-Handbook of Statistical Methods*. National Institute of Standards and Technology; 2019. Available from: <https://www.itl.nist.gov/div898/handbook/eda/section3/eda35b.htm>.
- [25] Engle RF. Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation. *Econometrica*. 1982 Jul;50(4):987–1007. Available from: <https://www.jstor.org/stable/1912773?seq=1>.
- [26] Sheppard K, Khrapov S, Lipták G, van Hattem R, mikedeltalima, Hammudoglu J, et al.. *bashtage/arch: Release 7.2*. Zenodo; 2024. Available from: <https://doi.org/10.5281/zenodo.14035889>.
- [27] Goodfellow I, Bengio Y, Courville A. *Deep Learning*. MIT Press; 2016. <http://www.deeplearningbook.org>.
- [28] Terven J, Cordova-Esparza DM, Romero-González JA, Ramírez-Pedraza A, Chávez-Urbiola EA. A comprehensive survey of loss functions and metrics in deep learning. *Artificial Intelligence Review*. 2025 Apr;58(7). Available from: <http://dx.doi.org/10.1007/s10462-025-11198-7>.
- [29] Jadon A, Patil A, Jadon S. A Comprehensive Survey of Regression Based Loss Functions for Time Series Forecasting; 2022. Available from: <https://arxiv.org/abs/2211.02989>.
- [30] Nielsen MA. *Neural Networks and Deep Learning*. Determination Press; 2019. Available from: <https://www.google.com/search?client=safari&rls=en&q=Michael+A.+Nielsen.+Neural+Networks+and+Deep+Learning.+Determination+Press%2C+2018.&ie=UTF-8&oe=UTF-8>.
- [31] Kingma DP, Ba J. Adam: A Method for Stochastic Optimization; 2017. Available from: <https://arxiv.org/abs/1412.6980>.
- [32] Mallya A. LSTM. Github; 2016. Available from: <https://arunmallya.github.io/writeups/nn/lstm/index.html#/7>.
- [33] Liu LY, Patton AJ, Sheppard K. Does anything beat 5-minute RV? A comparison of realized measures across multiple asset classes. *Journal of Econometrics*. 2015 Jul;187(1):293–311. Available from: <https://www.sciencedirect.com/science/article/pii/S0304407615000329>.
- [34] Patton AJ. Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics*. 2011 Jan;160(1):246–256. Available from: <https://www.sciencedirect.com/science/article/pii/S030440761000076X>.

- [35] Andersen TG, Bollerslev T. Answering the Skeptics: Yes, Standard Volatility Models do Provide Accurate Forecasts. *International Economic Review*. 1998 Nov;39(4):885. Available from: <http://www.jstor.org/stable/2527343%20>.
- [36] Ge W, Lalbakhsh P, Isai L, Lenskiy A, Suominen H. Neural Network–Based Financial Volatility Forecasting: A Systematic Review. *ACM Computing Surveys*. 2023 Jan;55(1):1–30. Available from: <https://doi.org/10.1145/3483596>.
- [37] Fuertes AM, Izzeldin M, Kalotychou E. On forecasting daily stock volatility: The role of intraday information and market conditions. *International Journal of Forecasting*. 2009 Apr;25(2):259–281. Available from: <https://www.sciencedirect.com/science/article/pii/S0169207009000077>.
- [38] Brooks C. *Introductory Econometrics for Finance*. 2nd ed. Cambridge University Press; 2008.
- [39] Kristjanpoller W, Fadic A, Minutolo MC. Volatility forecast using hybrid Neural Network models. *Expert Systems with Applications*. 2014 Apr;41(5):2437–2442. Available from: <https://www.sciencedirect.com/science/article/pii/S0957417413007975>.
- [40] Keskar NS, Mudigere D, Nocedal J, Smelyanskiy M, Tang PTP. On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima. *CoRR*. 2016;abs/1609.04836. Available from: <http://arxiv.org/abs/1609.04836>.
- [41] Srivastava N, Hinton G, Krizhevsky A, Sutskever I, Salakhutdinov R. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*. 2014;15(56):1929–58. Available from: <http://jmlr.org/papers/v15/srivastava14a.html>.
- [42] McMillan DG, Speight AEH. Daily volatility forecasts: reassessing the performance of GARCH models. *Journal of Forecasting*. 2004 Sep;23(6):449–460. Available from: <https://onlinelibrary.wiley.com/doi/epdf/10.1002/for.926>.
- [43] Hansen PR, Huang Z, Shek HH. Realized GARCH: a joint model for returns and realized measures of volatility. *Journal of Applied Econometrics*. 2011 Mar;27(6):877–906. Available from: <https://onlinelibrary.wiley.com/doi/full/10.1002/jae.1234>.
- [44] Liu C, Wang C, Tran MN, Kohn R. A long short-term memory enhanced realized conditional heteroskedasticity model. *Economic Modelling*. 2025 Jan;142:106922. Available from: <https://www.sciencedirect.com/science/article/pii/S0264999324002797>.
- [45] Xiong R, Nichols EP, Shen Y. Deep Learning Stock Volatility with Google Domestic Trends; 2016. Available from: <https://arxiv.org/abs/1512.04916>.